

操作系统面试题

 面试鸭 mianshiya.com

程序员面试 刷题神器

八股文一网打尽 面试从未如此轻松

- ✓ 9000+ 中大厂高频面试题
- ✓ 大厂面试官团队原创优质题解
- ✓ 网页版+小程序+IDE、VSCode插件
- ✓ 真题命中率高，持续更新

微信扫一扫 >>

刷高频题拿高薪offer



用户态和内核态

用户态和内核态的区别？

内核态和用户态是操作系统中的两种运行模式。它们的主要区别在于权限和可执行的操作：

- 内核态（Kernel Mode）：在内核态下，CPU可以执行所有的指令和访问所有的硬件资源。这种模式下的操作具有更高的权限，主要用于操作系统内核的运行。
- 用户态（User Mode）：在用户态下，CPU只能执行部分指令集，无法直接访问硬件资源。这种模式下的操作权限较低，主要用于运行用户程序。

内核态的底层操作主要包括：内存管理、进程管理、设备驱动程序控制、系统调用等。这些操作涉及到操作系统的核心功能，需要较高的权限来执行。

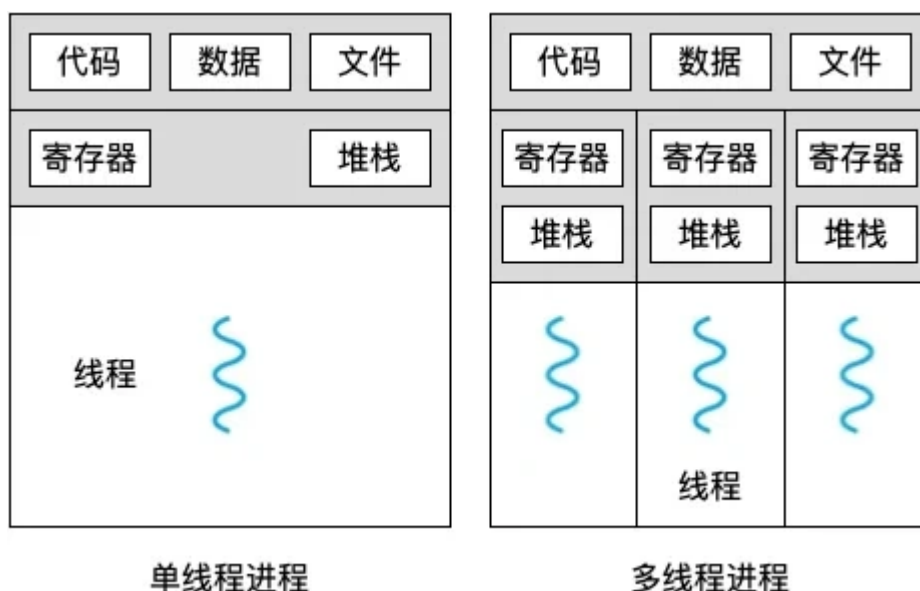
分为内核态和用户态的原因主要有以下几点：

- 安全性：通过对权限的划分，用户程序无法直接访问硬件资源，从而避免了恶意程序对系统资源的破坏。
- 稳定性：用户态程序出现问题时，不会影响到整个系统，避免了程序故障导致系统崩溃的风险。
- 隔离性：内核态和用户态的划分使得操作系统内核与用户程序之间有了明确的边界，有利于系统的模块化和维护。

内核态和用户态的划分有助于保证操作系统的安全性、稳定性和易维护性。

进程管理

线程和进程的区别是什么？



- **本质区别：**进程是操作系统资源分配的基本单位，而线程是任务调度和执行的基本单位
- **在开销方面：**每个进程都有独立的代码和数据空间（程序上下文），程序之间的切换会有较大的开销；线程可以看做轻量级的进程，同一类线程共享代码和数据空间，每个线程都有自己独立的运行栈和程序计数器（PC），线程之间切换的开销小
- **稳定性方面：**进程中某个线程如果崩溃了，可能会导致整个进程都崩溃。而进程中的子进程崩溃，并不会影响其他进程。
- **内存分配方面：**系统在运行的时候会为每个进程分配不同的内存空间；而对线程而言，除了CPU外，系统不会为线程分配内存（线程所使用的资源来自其所属进程的资源），线程组之间只能共享资源
- **包含关系：**没有线程的进程可以看做是单线程的，如果一个进程内有多线程，则执行过程不是一条线的，而是多条线

进程，线程，协程的区别是什么？

- 首先，我们来谈谈**进程**。进程是操作系统中进行资源分配和调度的基本单位，它拥有自己的独立内存空间和系统资源。每个进程都有独立的堆和栈，不与其他进程共享。进程间通信需要通过特定的机制，如管道、消息队列、信号量等。由于进程拥有独立的内存空间，因此其稳定性和安全性相对较高，但同时上下文切换的开销也较大，因为需要保存和恢复整个进程的状态。
- 接下来是**线程**。线程是进程内的一个执行单元，也是CPU调度和分派的基本单位。与进程不同，线程共享进程的内存空间，包括堆和全局变量。线程之间通信更加高效，因为它们可以直接读写共享内存。线程的上下文切换开销较小，因为只需要保存和恢复线程的上下文，而不是整个进程的状态。然而，由于多个线程共享内存空间，因此存在数据竞争和线程安全的问题，需要通过同步和互斥机制来解决。
- 最后是**协程**。协程是一种用户态的轻量级线程，其调度完全由用户程序控制，而不需要内核的参与。协程拥有自己的寄存器上下文和栈，但与其他协程共享堆内存。协程的切换开销非常小，因为只需要保存和恢复协程的上下文，而无需进行内核级的上下文切换。这使得协程在处理大量并发任务时具有非常高的效率。然而，协程需要程序员显式地进行调度和管理，相对于线程和进程来说，其编程模型更为复杂。

为什么进程崩溃不会对其他进程产生很大影响

主要是因为：

- **进程隔离性：**每个进程都有自己独立的内存空间，当一个进程崩溃时，其内存空间会被操作系统回收，不会影响其他进程的内存空间。这种进程间的隔离性保证了一个进程崩溃不会直接影响其他进程的执行。
- **进程独立性：**每个进程都是独立运行的，它们之间不会共享资源，如文件、网络连接等。因此，一个进程的崩溃通常不会对其他进程的资源产生影响。

你说到进程是分配资源的基本单位，那么这个资源指的是什么？

虚拟内存、文件句柄、信号量等资源。

讲下为什么进程之下还要设计线程？

我们举个例子，假设你要编写一个视频播放器软件，那么该软件功能的核心模块有三个：

- 从视频文件当中读取数据；
- 对读取的数据进行解压缩；
- 把解压缩后的视频数据播放出来；

对于单进程的实现方式，我想大家都会是以下这个方式：

```
main()
{
    while(1)
    {
        // 读取文件数据
        Read();

        // 解压缩数据
        Decompress();

        // 播放解压缩数据的数据
        Play();
    }
}
```

I/O → Read();

CPU → Decompress();

对于单进程的这种方式，存在以下问题：

- 播放出来的画面和声音会不连贯，因为当 CPU 能力不够强的时候，Read 的时候可能进程就等在这了，这样就会导致等半天才进行数据解压和播放；
- 各个函数之间不是并发执行，影响资源的使用效率；

那改进成多进程的方式：

```
main()
{
    while(1)
    {
        // 读取文件数据
        Read();
    }
}
```

```
main()
{
    while(1)
    {
        // 解压缩数据
        Decompress();
    }
}
```

```
main()
{
    while(1)
    {
        // 播放解压缩数据的数据
        Play();
    }
}
```

对于多进程的这种方式，依然会存在问题：

- 进程之间如何通信，共享数据？
- 维护进程的系统开销较大，如创建进程时，分配资源、建立 PCB；终止进程时，回收资源、撤销 PCB；进程切换时，保存当前进程的状态信息；

那到底如何解决呢？需要有一种新的实体，满足以下特性：

- 实体之间可以并发运行；
- 实体之间共享相同的地址空间；

这个新的实体，就是**线程(Thread)**，线程之间可以并发运行且共享相同的地址空间。

多线程比单线程的优势，劣势？

- 多线程比单线程的**优势**：提高程序的运行效率，可以充分利用多核处理器的资源，同时处理多个任务，加快程序的执行速度。
- 多线程比单线程的**劣势**：存在多线程数据竞争访问的问题，需要通过锁机制来保证线程安全，增加了加锁的开销，并且还会有死锁的风险。多线程会消耗更多系统资源，如CPU和内存，因为每个线程都需要占用一定的内存和处理时间。

多线程是不是越多越好，太多会有什么问题？

多线程不一定越多越好，过多的线程可能会导致一些问题。

- 切换开销：线程的创建和切换会消耗系统资源，包括内存和CPU。如果创建太多线程，会占用大量的系统资源，导致系统负载过高，某个线程崩溃后，可能会导致进程崩溃。
- 死锁的问题：过多的线程可能会导致竞争条件和死锁。竞争条件指的是多个线程同时访问和修改共享资源，如果没有合适的同步机制，可能会导致数据不一致或错误的结果。而死锁则是指多个线程相互等待对方释放资源，导致程序无法继续执行。

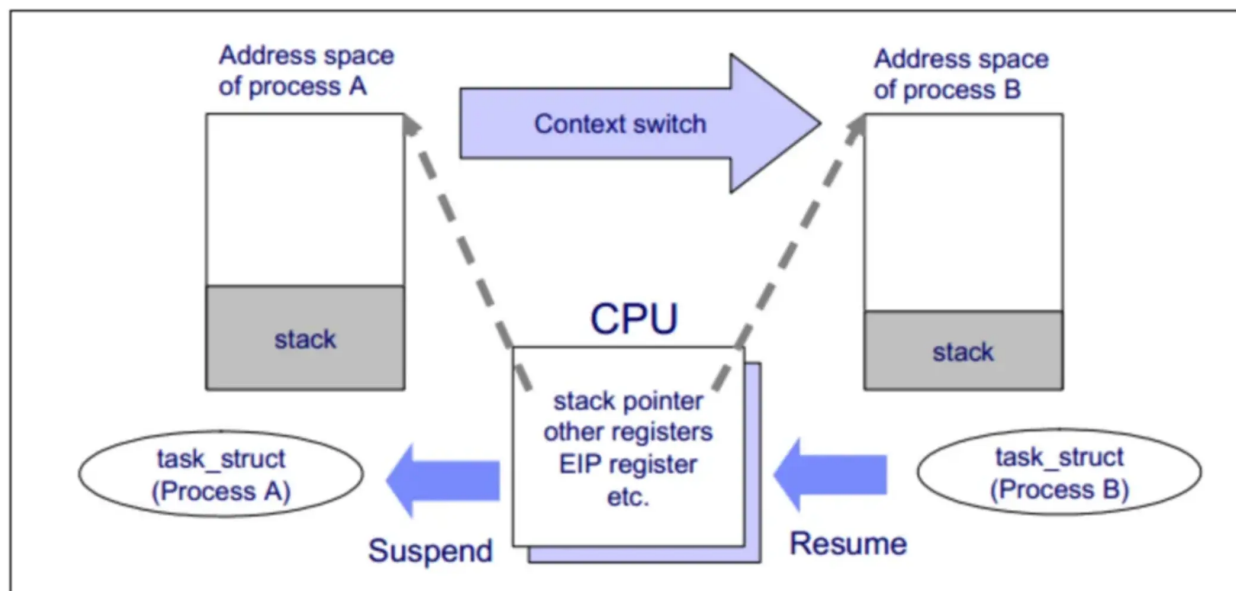
进程切换和线程切换的区别？

1. 进程切换：进程切换涉及到更多的内容，包括整个进程的地址空间、全局变量、文件描述符等。因此，进程切换的开销通常比线程切换大。
2. 线程切换：线程切换只涉及到线程的堆栈、寄存器和程序计数器等，不涉及进程级别的资源，因此线程切换的开销较小。

线程切换为什么比进程切换快，节省了什么资源？

线程切换比进程切换快是因为线程共享同一进程的地址空间和资源，线程切换时只需切换堆栈和程序计数器等少量信息，而不需要切换地址空间，避免了进程切换时需要切换内存映射表等大量资源的开销，从而节省了时间和系统资源。

线程切换详细过程是怎么样的？上下文保存在哪里？



线程切换的详细过程可以分为以下几个步骤：

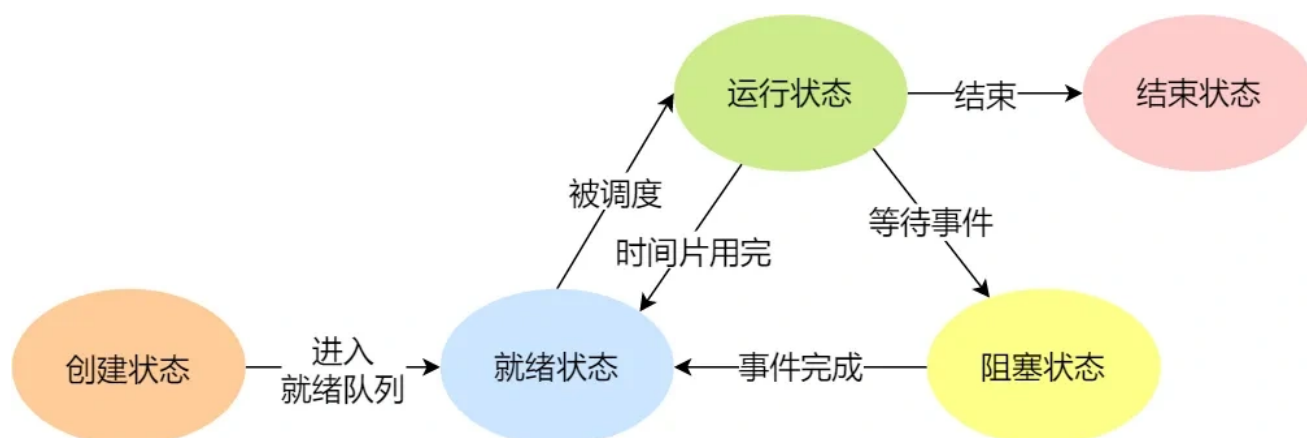
- 上下文保存：当操作系统决定切换到另一个线程时，它首先会保存当前线程的上下文信息。上下文信息包括寄存器状态、程序计数器、堆栈指针等，用于保存线程的执行状态。
- 切换到调度器：操作系统将执行权切换到调度器（Scheduler）。调度器负责选择下一个要执行的线程，并根据调度算法做出决策。
- 上下文恢复：调度器选择了下一个要执行的线程后，它会从该线程保存的上下文信息中恢复线程的执行状态。
- 切换到新线程：调度器将执行权切换到新线程，使其开始执行。

上下文信息的保存通常由操作系统负责管理，具体保存在哪里取决于操作系统的实现方式。一般情况下，上下文信息会保存在线程的控制块（Thread Control Block, TCB）中。

TCB是操作系统用于管理线程的数据结构，包含了线程的状态、寄存器的值、堆栈信息等。当发生线程切换时，操作系统会通过切换TCB来保存和恢复线程的上下文信息。

进程的状态（五种状态），如何切换？

一个完整的进程状态的变迁如下图：



进程五种状态的变迁

再来详细说明一下进程的状态变迁：

- *NULL -> 创建状态*: 一个新进程被创建时的第一个状态;
- *创建状态 -> 就绪状态*: 当进程被创建完成并初始化后, 一切就绪准备运行时, 变为就绪状态, 这个过程是很快;
- *就绪状态 -> 运行状态*: 处于就绪状态的进程被操作系统的进程调度器选中后, 就分配给 CPU 正式运行该进程;
- *运行状态 -> 结束状态*: 当进程已经运行完成或出错时, 会被操作系统作结束状态处理;
- *运行状态 -> 就绪状态*: 处于运行状态的进程在运行过程中, 由于分配给它的运行时间片用完, 操作系统会把该进程变为就绪态, 接着从就绪态选中另外一个进程运行;
- *运行状态 -> 阻塞状态*: 当进程请求某个事件且必须等待时, 例如请求 I/O 事件;
- *阻塞状态 -> 就绪状态*: 当进程要等待的事件完成时, 它从阻塞状态变到就绪状态;

进程上下文有哪些?

各个进程之间是共享 CPU 资源的, 在不同的时候进程之间需要切换, 让不同的进程可以在 CPU 执行, 那么这~~个~~**一个进程切换到另一个进程运行, 称为进程的上下文切换。**

在详细说进程上下文切换前, 我们先来看看 CPU 上下文切换

大多数操作系统都是多任务, 通常支持大于 CPU 数量的任务同时运行。实际上, 这些任务并不是同时运行的, 只是因为系统在很短的时间内, 让各个任务分别在 CPU 运行, 于是就造成同时运行的错觉。

任务是交给 CPU 运行的, 那么在每个任务运行前, CPU 需要知道任务从哪里加载, 又从哪里开始运行。

所以, 操作系统需要事先帮 CPU 设置好 **CPU 寄存器和程序计数器**。

CPU 寄存器是 CPU 内部一个容量小, 但是速度极快的内存 (缓存)。我举个例子, 寄存器像是你的口袋, 内存像你的书包, 硬盘则是你家里的柜子, 如果你的东西存放到口袋, 那肯定是你从书包或家里柜子取出来要快的多。

再来, 程序计数器则是用来存储 CPU 正在执行的指令位置、或者即将执行的下一条指令位置。

所以说, CPU 寄存器和程序计数是 CPU 在运行任何任务前, 所必须依赖的环境, 这些环境就叫做 **CPU 上下文**。

既然知道了什么是 CPU 上下文, 那理解 CPU 上下文切换就不难了。

CPU 上下文切换就是先把前一个任务的 CPU 上下文 (CPU 寄存器和程序计数器) 保存起来, 然后加载新任务的上下文到这些寄存器和程序计数器, 最后再跳转到程序计数器所指的新位置, 运行新任务。

系统内核会存储保持下来的上下文信息, 当此任务再次被分配给 CPU 运行时, CPU 会重新加载这些上下文, 这样就能保证任务原来的状态不受影响, 让任务看起来还是连续运行。

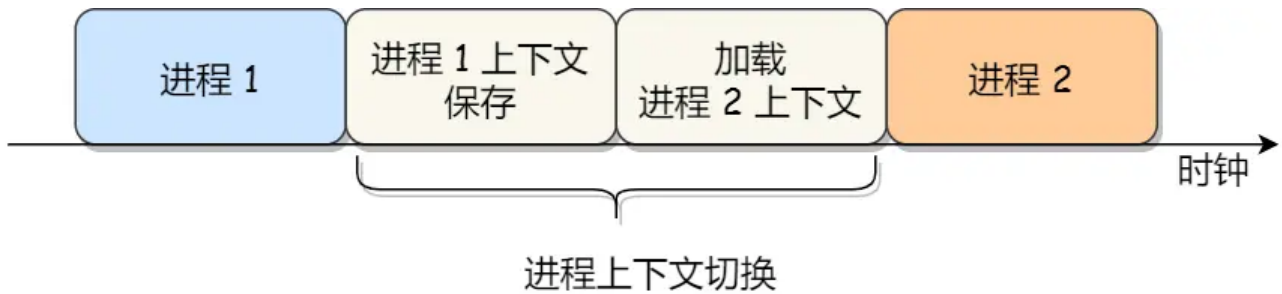
上面说到所谓的「任务」, 主要包含进程、线程和中断。所以, 可以根据任务的不同, 把 CPU 上下文切换分成:**进程上下文切换、线程上下文切换和中断上下文切换。**

进程的上下文切换到底是切换什么呢?

进程是由内核管理和调度的, 所以进程的切换只能发生在内核态。

所以, **进程的上下文切换不仅包含了虚拟内存、栈、全局变量等用户空间的资源, 还包括了内核堆栈、寄存器等内核空间的资源。**

通常, 会把交换的信息保存在进程的 PCB, 当要运行另外一个进程的时候, 我们需要从这个进程的 PCB 取出上下文, 然后恢复到 CPU 中, 这使得这个进程可以继续执行, 如下图所示:

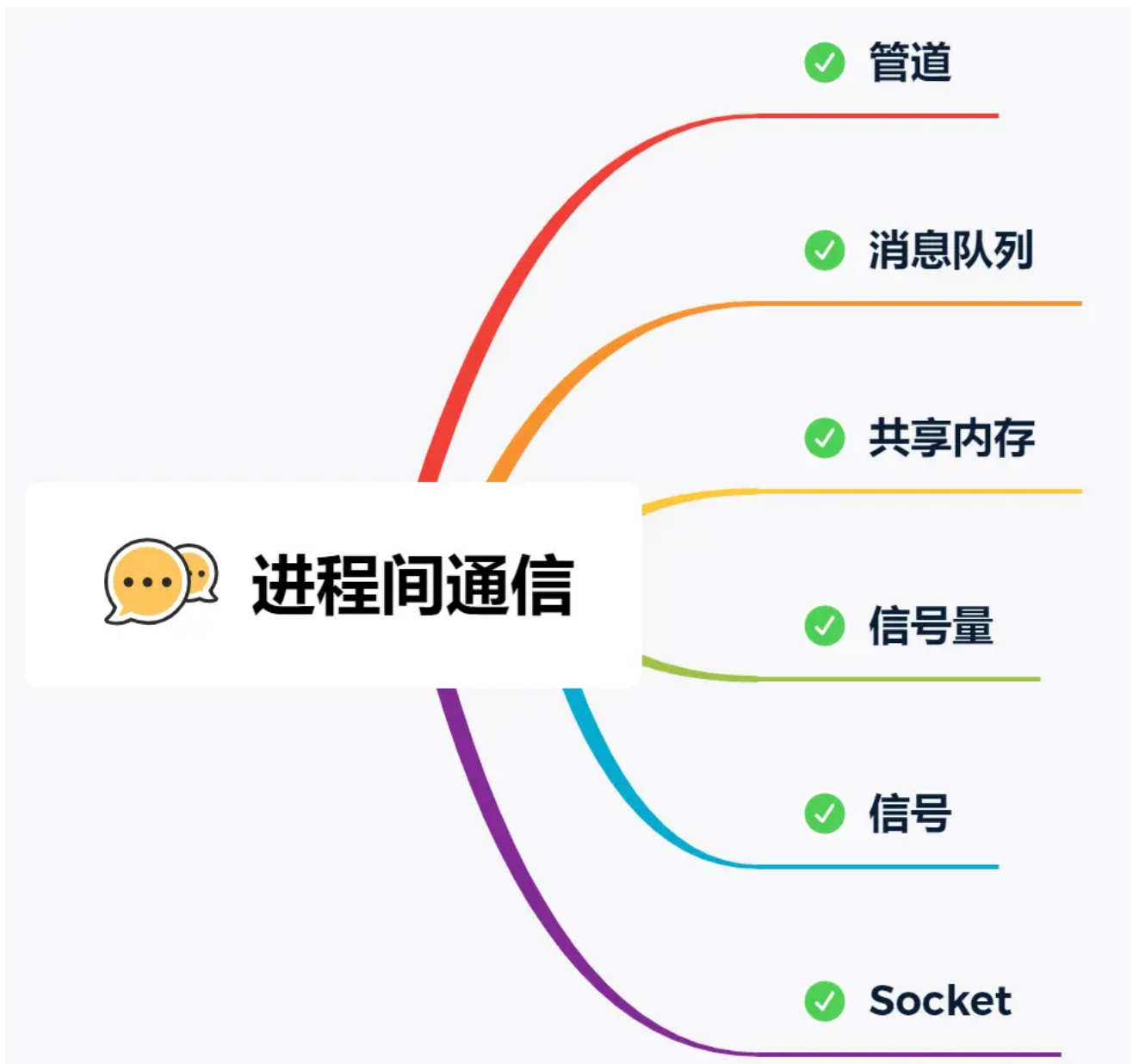


大家需要注意，进程的上下文开销是很关键的，我们希望它的开销越小越好，这样可以使得进程可以把更多时间花费在执行程序上，而不是耗费在上下文切换。

进程间通讯有哪些方式？

Linux 内核提供了不少进程间通信的方式：

- 管道
- 消息队列
- 共享内存
- 信号
- 信号量
- socket



Linux 内核提供了不少进程间通信的方式，其中最简单的方式就是管道，管道分为「匿名管道」和「命名管道」。

匿名管道顾名思义，它没有名字标识，匿名管道是特殊文件只存在于内存，没有存在于文件系统中，shell 命令中的「|」竖线就是匿名管道，通信的数据是**无格式的流并且大小受限**，通信的方式是**单向的**，数据只能在一个方向上流动，如果要双向通信，需要创建两个管道，再来**匿名管道是只能用于存在父子关系的进程间通信**，匿名管道的生命周期随着进程创建而建立，随着进程终止而消失。

命名管道突破了匿名管道只能在亲缘关系进程间的通信限制，因为使用命名管道的前提，需要在文件系统创建一个类型为 p 的设备文件，那么毫无关系的进程就可以通过这个设备文件进行通信。另外，不管是匿名管道还是命名管道，进程写入的数据都是**缓存在内核**中，另一个进程读取数据时候自然也是从内核中获取，同时通信数据都遵循**先进先出**原则，不支持 lseek 之类的文件定位操作。

消息队列克服了管道通信的数据是无格式的字节流的问题，消息队列实际上是保存在内核的「消息链表」，消息队列的消息体是可以用户自定义的数据类型，发送数据时，会被分成一个一个独立的消息体，当然接收数据时，也要与发送方发送的消息体的数据类型保持一致，这样才能保证读取的数据是正确的。消息队列通信的速度不是最及时的，毕竟**每次数据的写入和读取都需要经过用户态与内核态之间的拷贝过程**。

共享内存可以解决消息队列通信中用户态与内核态之间数据拷贝过程带来的开销，它直接分配一个共享空间，每个进程都可以直接访问，就像访问进程自己的空间一样快捷方便，不需要陷入内核态或者系统调用，大大提高了通信的速度，享有**最快的进程间通信方式**之名。但是便捷高效的共享内存通信，带来新的问题，多进程竞争同个共享资源会造成数据的错乱。

那么，就需要**信号量**来保护共享资源，以确保任何时刻只能有一个进程访问共享资源，这种方式就是互斥访问。信号量不仅可以实现访问的互斥性，还可以实现进程间的同步，信号量其实是一个计数器，表示的是资源个数，其值可以通过两个原子操作来控制，分别是 **P 操作**和 **V 操作**。

与信号量名字很相似的叫**信号**，它俩名字虽然相似，但功能一点儿都不一样。信号是**异步通信机制**，信号可以在应用进程和内核之间直接交互，内核也可以利用信号来通知用户空间的进程发生了哪些系统事件，信号事件的来源主要有硬件来源（如键盘 Ctrl+C）和软件来源（如 kill 命令），一旦有信号发生，**进程有三种方式响应信号 1. 执行默认操作、2. 捕捉信号、3. 忽略信号**。有两个信号是应用进程无法捕捉和忽略的，即 SIGKILL 和 SIGSTOP，这是为了方便我们能在任何时候结束或停止某个进程。

前面说到的通信机制，都是工作于同一台主机，如果**要与不同主机的进程间通信，那么就需要 Socket 通信了**。Socket 实际上不仅用于不同的主机进程间通信，还可以用于本地主机进程间通信，可根据创建 Socket 的类型不同，分为三种常见的通信方式，一个是基于 TCP 协议的通信方式，一个是基于 UDP 协议的通信方式，一个是本地进程间通信方式。

管道有几种方式？

管道在Linux中有两种方式：匿名管道和命名管道。

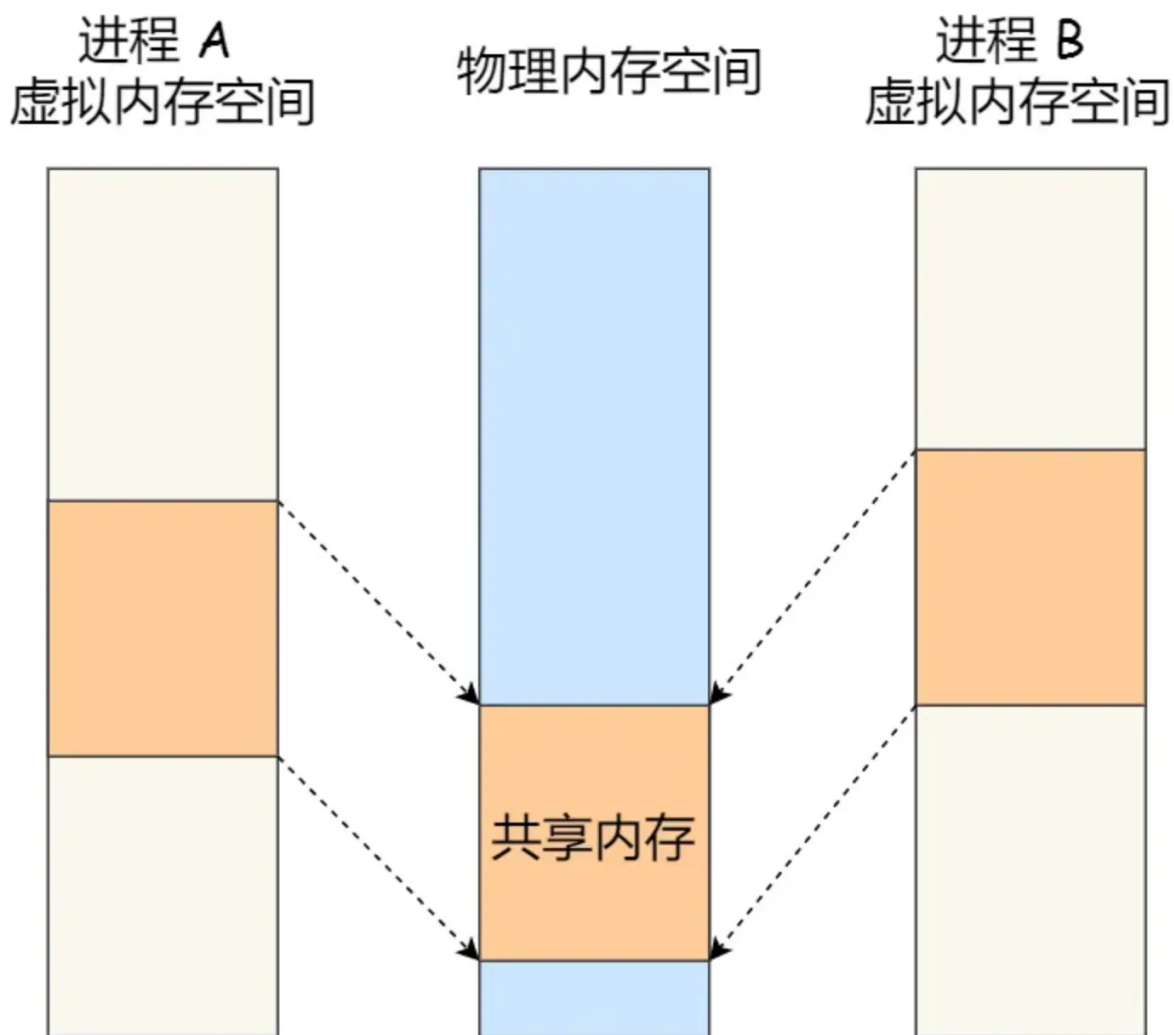
- 匿名管道：是一种在父子进程或者兄弟进程之间进行通信的机制，只能用于具有亲缘关系的进程间通信，通常通过pipe系统调用创建。
- 命名管道：是一种允许无关的进程间进行通信的机制，基于文件系统，可以在不相关的进程之间进行通信。

信号和信号量有什么区别？

- 信号：一种处理异步事件的方式。信号是比较复杂的通信方式，用于通知接收进程有某种事件发生，除了用于进程外，还可以发送信号给进程本身。
- 信号量：进程间通信处理同步互斥的机制。是在多线程环境下使用的一种设施，它负责协调各个线程，以保证它们能够正确，合理的使用公共资源。

共享内存怎么实现的？

共享内存的机制，就是拿出一块虚拟地址空间来，映射到相同的物理内存中。这样这个进程写入的东西，另外一个进程马上就能看到了，都不需要拷贝来拷贝去，传来传去，大大提高了进程间通信的速度。



线程间通讯有什么方式？

Linux系统提供了五种用于线程通信的方式：**互斥锁、读写锁、条件变量、自旋锁和信号量。**

- **互斥锁 (Mutex)**：互斥量(mutex)从本质上说是一把锁，在访问共享资源前对互斥量进行加锁，在访问完成后释放互斥量上的锁。对互斥量进行加锁以后，任何其他试图再次对互斥锁加锁的线程将会阻塞直到当前线程释放该互斥锁。如果释放互斥锁时有多个线程阻塞，所有在该互斥锁上的阻塞线程都会变成可运行状态，第一个变为运行状态的线程可以对互斥锁加锁，其他线程将会看到互斥锁依然被锁住，只能回去再次等待它重新变为可用。
- **条件变量 (Condition Variables)**：条件变量(cond)是在多线程程序中用来实现"等待--》唤醒"逻辑常用的方法。条件变量利用线程间共享的全局变量进行同步的一种机制，主要包括两个动作：一个线程等待"条件变量的条件成立"而挂起；另一个线程使"条件成立"。为了防止竞争，条件变量的使用总是和一个互斥锁结合在一起。线程在改变条件状态前必须首先锁住互斥量，函数pthread_cond_wait把自己放到等待条件的线程列表上，然后对互斥锁解锁(这两个操作是原子操作)。在函数返回时，互斥量再次被锁住。
- **自旋锁 (Spinlock)**：自旋锁通过 CPU 提供的 CAS 函数 (Compare And Swap)，在「用户态」完成加锁和解锁操作，不会主动产生线程上下文切换，所以相比互斥锁来说，会快一些，开销也小一些。一般加锁的过程，包含两个步骤：第一步，查看锁的状态，如果锁是空闲的，则执行第二步；第二步，将锁设置为当前线程持有；使用自旋锁的时候，当发生多线程竞争锁的情况，加锁失败的线程会「忙等待」，直到它拿到锁。CAS 函数就把这两个步骤合并成一条硬件级指令，形成**原子指令**，这样就保证了这两个步骤是不可分割的，要么一次性执行完两个步骤，要么两个步骤都不执行。这里的「忙等待」可以用 while 循环等待实现，不过最好是使用 CPU 提供的 PAUSE 指令来实现「忙等待」，因为可以减少循环等待时的耗电量。

- **信号量 (Semaphores)**：信号量可以是命名的（有名信号量）或无名的（仅限于当前进程内的线程），用于控制对资源的访问次数。通常**信号量表示资源的数量**，对应的变量是一个整型（sem）变量。另外，还有**两个原子操作的系统调用函数来控制信号量的**，分别是：**P 操作**：将 sem 减 1，相减后，如果 $\text{sem} < 0$ ，则进程/线程进入阻塞等待，否则继续，表明 P 操作可能会阻塞；**V 操作**：将 sem 加 1，相加后，如果 $\text{sem} \leq 0$ ，唤醒一个等待中的进程/线程，表明 V 操作不会阻塞；
- **读写锁 (Read-Write Locks)**：读写锁从字面意思我们也可以知道，它由「读锁」和「写锁」两部分构成，如果只读取共享资源用「读锁」加锁，如果要修改共享资源则用「写锁」加锁。所以，**读写锁适用于能明确区分读操作和写操作的场景**。读写锁的工作原理是：当「写锁」没有被线程持有时，多个线程能够并发地持有读锁，这大大提高了共享资源的访问效率，因为「读锁」是用于读取共享资源的场景，所以多个线程同时持有读锁也不会破坏共享资源的数据。但是，一旦「写锁」被线程持有后，读线程的获取读锁的操作会被阻塞，而且其他写线程的获取写锁的操作也会被阻塞。所以说，写锁是独占锁，因为任何时刻只能有一个线程持有写锁，类似互斥锁和自旋锁，而读锁是共享锁，因为读锁可以被多个线程同时持有。知道了读写锁的工作原理后，我们可以发现，**读写锁在读多写少的场景，能发挥出优势**。

除了互斥锁你还知道什么锁？分别应用于什么场景？

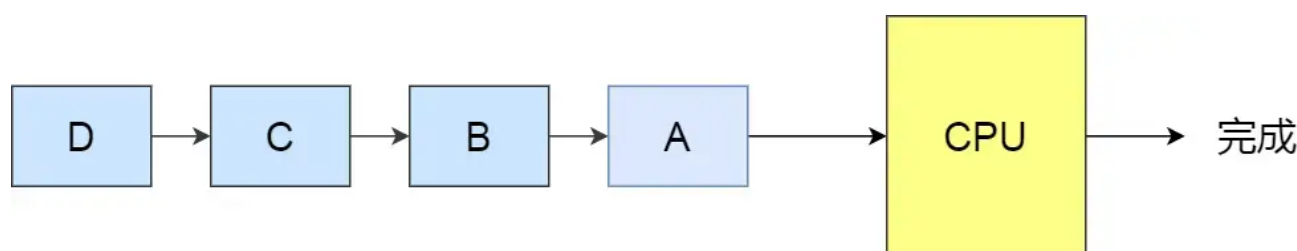
还有读写锁、自旋锁、条件变量、信号量。

1. 读写锁：读写锁允许多个线程同时读取共享资源，但只允许一个线程进行写操作。适用于读操作频繁、写操作较少的场景，可以提高并发性能。
2. 自旋锁：自旋锁是一种忙等待锁，线程在获取锁时不会进入阻塞状态，而是循环忙等待直到获取到锁。适用于临界区很小且锁的持有时间很短的场景，避免线程频繁切换带来的开销。
3. 条件变量：条件变量用于线程间的同步和通信。它通常与互斥锁一起使用，线程可以通过条件变量等待某个条件满足，当条件满足时，其他线程可以通过条件变量发送信号通知等待线程。
4. 信号量：信号量是一种计数器，用于控制对共享资源的访问。它可以用来限制同时访问资源的线程数量，或者用于线程间的同步。

进程调度算法有哪些？

先来先服务调度算法

最简单的一个调度算法，就是非抢占式的**先来先服务 (*First Come First Severd, FCFS*)** 算法了。

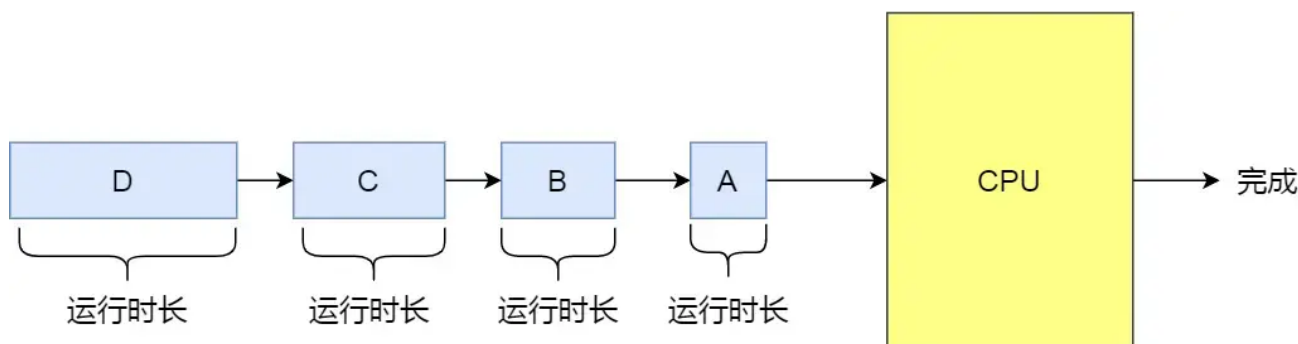


顾名思义，先来后到，**每次从就绪队列选择最先进入队列的进程**，然后一直运行，直到进程退出或被阻塞，才会继续从队列中选择第一个进程接着运行。

这似乎很公平，但是当长作业先运行了，那么后面的短作业等待的时间就会很长，不利于短作业。FCFS 对长作业有利，适用于 CPU 繁忙型作业的系统，而不适用于 I/O 繁忙型作业的系统。

最短作业优先调度算法

最短作业优先 (*Shortest Job First, SJF*) 调度算法同样也是顾名思义，它会**优先选择运行时间最短的进程来运行**，这有助于提高系统的吞吐量。



这显然对长作业不利，很容易造成一种极端现象。

比如，一个长作业在就绪队列等待运行，而这个就绪队列有非常多的短作业，那么就会使得长作业不断的往后推，周转时间变长，致使长作业长期不会被运行。

高响应比优先调度算法

前面的「先来先服务调度算法」和「最短作业优先调度算法」都没有很好的权衡短作业和长作业。

那么，**高响应比优先 (*Highest Response Ratio Next, HRRN*) 调度算法**主要是权衡了短作业和长作业。

每次进行进程调度时，先计算「响应比优先级」，然后把「响应比优先级」最高的进程投入运行，「响应比优先级」的计算公式：

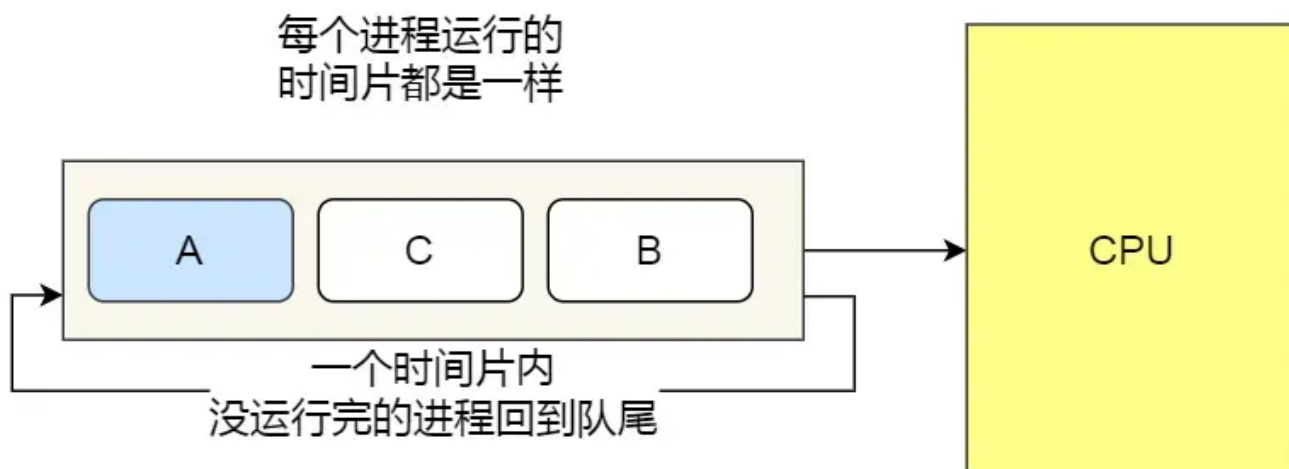
$$\text{优先权} = \frac{\text{等待时间} + \text{要求服务时间}}{\text{要求服务时间}}$$

从上面的公式，可以发现：

- 如果两个进程的「等待时间」相同时，「要求的服务时间」越短，「响应比」就越高，这样短作业的进程容易被选中运行；
- 如果两个进程「要求的服务时间」相同时，「等待时间」越长，「响应比」就越高，这就兼顾到了长作业进程，因为进程的响应比可以随时间等待的增加而提高，当其等待时间足够长时，其响应比便可以升到很高，从而获得运行的机会；

时间片轮转调度算法

最古老、最简单、最公平且使用最广的算法就是**时间片轮转 (*Round Robin, RR*) 调度算法**。



每个进程被分配一个时间段，称为时间片 (*Quantum*)，即允许该进程在该时间段中运行。

- 如果时间片用完，进程还在运行，那么将会把此进程从 CPU 释放出来，并把 CPU 分配另外一个进程；
- 如果该进程在时间片结束前阻塞或结束，则 CPU 立即进行切换；

另外，时间片的长度就是一个很关键的点：

- 如果时间片设得太短会导致过多的进程上下文切换，降低了 CPU 效率；
- 如果设得太长又可能引起对短作业进程的响应时间变长。将

通常时间片设为 20ms~50ms 通常是一个比较合理的折中值。

最高优先级调度算法

前面的「时间片轮转算法」做了个假设，即让所有的进程同等重要，也不偏袒谁，大家的运行时间都一样。

但是，对于多用户计算机系统就有不同的看法了，它们希望调度是有优先级的，即希望调度程序能**从就绪队列中选择最高优先级的进程进行运行，这称为最高优先级 (*Highest Priority First, HPF*) 调度算法**。进程的优先级可以分为，静态优先级或动态优先级：

- 静态优先级：创建进程时候，就已经确定了优先级了，然后整个运行时间优先级都不会变化；
- 动态优先级：根据进程的动态变化调整优先级，比如如果进程运行时间增加，则降低其优先级，如果进程等待时间（就绪队列的等待时间）增加，则升高其优先级，也就是**随着时间的推移增加等待进程的优先级**。

该算法也有两种处理优先级高的方法，非抢占式和抢占式：

- 非抢占式：当就绪队列中出现优先级高的进程，运行完当前进程，再选择优先级高的进程。
- 抢占式：当就绪队列中出现优先级高的进程，当前进程挂起，调度优先级高的进程运行。

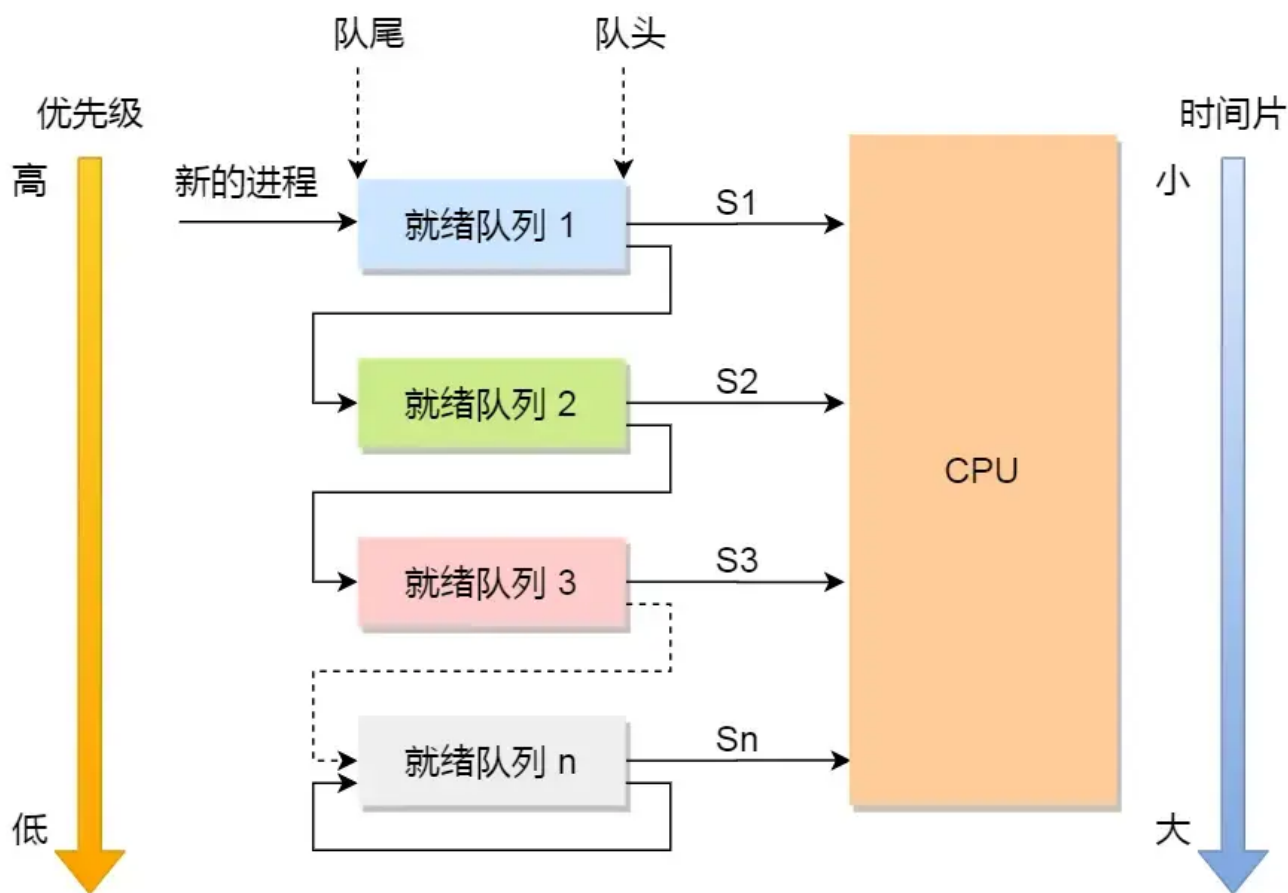
但是依然有缺点，可能会导致低优先级的进程永远不会运行。

多级反馈队列调度算法

多级反馈队列 (*Multilevel Feedback Queue*) 调度算法是「时间片轮转算法」和「最高优先级算法」的综合和发展。

顾名思义：

- 「多级」表示有多个队列，每个队列优先级从高到低，同时优先级越高时间片越短。
- 「反馈」表示如果有新的进程加入优先级高的队列时，立刻停止当前正在运行的进程，转而去运行优先级高的队列；



(时间片 : $S1 < S2 < S3$)

来看看，它是如何工作的：

- 设置了多个队列，赋予每个队列不同的优先级，每个队列优先级从高到低，同时优先级越高时间片越短；
- 新的进程会被放入到第一级队列的末尾，按先来先服务的原则排队等待被调度，如果在第一级队列规定的时间片没运行完成，则将其转入到第二级队列的末尾，以此类推，直至完成；
- 当较高优先级的队列为空，才调度较低优先级的队列中的进程运行。如果进程运行时，有新进程进入较高优先级的队列，则停止当前运行的进程并将其移入到原队列末尾，接着让较高优先级的进程运行；

可以发现，对于短作业可能可以在第一级队列很快被处理完。

对于长作业，如果在第一级队列处理不完，可以移入下次队列等待被执行，虽然等待的时间变长了，但是运行时间也会更长了，所以该算法很好的兼顾了长短作业，同时有较好的响应时间。

锁

为什么并发执行线程要加锁？

并发执行线程需要加锁主要是为了保护共享数据，防止出现"竞态条件"。

"竞态条件"是指当多个线程同时访问和操作同一块数据时，最终结果依赖于线程的执行顺序，这可能导致数据的不一致性。

通过加锁，我们可以确保在任何时刻只有一个线程能够访问共享数据，从而避免"竞态条件"，确保数据的一致性和完整性。

自旋锁是什么？应用在哪些场景？

自旋锁加锁失败后，线程会**忙等待**，直到它拿到锁。

自旋锁是通过 CPU 提供的 `CAS` 函数 (*Compare And Swap*)，在「用户态」完成加锁和解锁操作，不会主动产生线程上下文切换，所以相比互斥锁来说，会快一些，开销也小一些。

一般加锁的过程，包含两个步骤：

- 第一步，查看锁的状态，如果锁是空闲的，则执行第二步；
- 第二步，将锁设置为当前线程持有；

`CAS` 函数就把这两个步骤合并成一条硬件级指令，形成**原子指令**，这样就保证了这两个步骤是不可分割的，要么一次性执行完两个步骤，要么两个步骤都不执行。

比如，设锁为变量 `lock`，整数 0 表示锁是空闲状态，整数 `pid` 表示线程 ID，那么 `CAS(lock, 0, pid)` 就表示自旋锁的加锁操作，`CAS(lock, pid, 0)` 则表示解锁操作。

使用自旋锁的时候，当发生多线程竞争锁的情况，加锁失败的线程会「忙等待」，直到它拿到锁。这里的「忙等待」可以用 `while` 循环等待实现，不过最好是使用 CPU 提供的 `PAUSE` 指令来实现「忙等待」，因为可以减少循环等待时的耗电量。

自旋锁是最比较简单的一种锁，一直自旋，利用 CPU 周期，直到锁可用。**需要注意，在单核 CPU 上，需要抢占式的调度器（即不断通过时钟中断一个线程，运行其他线程）。否则，自旋锁在单 CPU 上无法使用，因为一个自旋的线程永远不会放弃 CPU。**

自旋锁开销少，在多核系统下一般不会主动产生线程切换，适合异步、协程等在用户态切换请求的编程方式，但如果被锁住的代码执行时间过长，自旋的线程会长时间占用 CPU 资源，所以自旋的时间和被锁住的代码执行的时间是成「正比」的关系，我们需要清楚的知道这一点。

自旋锁与互斥锁使用层面比较相似，但实现层面上完全不同：**当加锁失败时，互斥锁用「线程切换」来应对，自旋锁则用「忙等待」来应对。**

如果你能确定被锁住的代码执行时间很短，就不应该用互斥锁，而应该选用自旋锁，否则使用互斥锁。

死锁发生条件是什么？

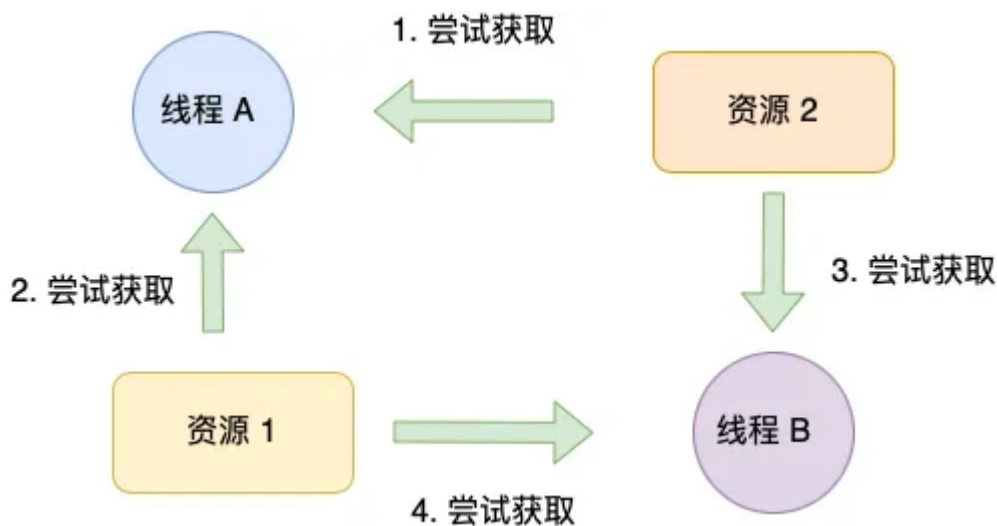
死锁只有**同时满足**以下四个条件才会发生：

- 互斥条件：互斥条件是指**多个线程不能同时使用同一个资源**。
- 持有并等待条件：持有并等待条件是指，当线程 A 已经持有了资源 1，又想申请资源 2，而资源 2 已经被线程 C 持有了，所以线程 A 就会处于等待状态，但是**线程 A 在等待资源 2 的同时并不会释放自己已经持有的资源 1**。
- 不可剥夺条件：不可剥夺条件是指，当线程已经持有了资源，**在自己使用完之前不能被其他线程获取**，线程 B 如果也想使用此资源，则只能在线程 A 使用完并释放后才能获取。
- 环路等待条件：环路等待条件指的是，在死锁发生的时候，**两个线程获取资源的顺序构成了环形链**。

如何避免死锁？

避免死锁问题就只需要破坏其中一个条件就可以，最常见的并且可行的就是**使用资源有序分配法，来破坏环路等待条件**。

那什么是资源有序分配法呢？线程 A 和 线程 B 获取资源的顺序要一样，当线程 A 是先尝试获取资源 A，然后尝试获取资源 B 的时候，线程 B 同样也是先尝试获取资源 A，然后尝试获取资源 B。也就是说，线程 A 和 线程 B 总是以相同的顺序申请自己想要的资源。



讲一下银行家算法

系统发生死锁是很正常的，我们需要主动去预防死锁，即进行有序的资源分配，使用**银行家算法**。

银行家算法是最有代表性的避免死锁的算法。

为什么叫银行家算法呢？就是这个算法的逻辑**很像银行放贷的逻辑，也就是尽可能避免坏账的出现。**

银行家算法的业务逻辑如下。

- **不负荷执行**：一个进程的最大需求量不超过系统拥有的总资源数，才会被接纳执行。
- **可分期**：一个进程可以分期请求资源，但总请求书不可超过最大需求量。
- **推迟分配**：当系统现有资源数小于进程需求时，对进程的需求可以延迟分配，但总让进程在有限时间内获取资源。

听起来有点绕，我们还是举个例子来说明。

假如系统中有三类互斥资源 R1、R2、R3，可用资源数分别是 9、8、5，在指定时刻有 P1、P2、P3、P4 和 P5 这五个进程，这些进程的对三类互斥资源的最大需求量和已分配资源数如下表所示，那么系统如何先后运行这五个进程，不会发生死锁问题？

进程	最大需求量（分别为R1 R2 R3）	已分配资源数（分别为R1 R2 R3）
P1	6 5 2	1 2 1
P2	2 2 1	2 1 1
P3	8 1 1	2 1 0
P4	1 2 1	1 2 0
P5	3 4 4	1 1 3

第一步：分析

首先分析首次需求的资源，**系统剩余可用资源数分别是 2、1、0**，各进程需要的资源数如下表所示。

资源 R1 的剩余可用资源数 = $9 - 1 - 2 - 2 - 1 - 1 = 2$ 。

资源 R2 的剩余可用资源数 = $8 - 2 - 1 - 1 - 2 - 1 = 1$ 。

资源 R3 的剩余可用资源数 = $5 - 1 - 1 - 0 - 0 - 3 = 0$ 。

进程	最大需求量	已分配资源数	首次分配需要的资源数
P1	6 5 2	1 2 1	5 3 1
P2	2 2 1	2 1 1	0 1 0
P3	8 1 1	2 1 0	6 0 1
P4	1 2 1	1 2 0	0 0 1
P5	3 4 4	1 1 3	2 3 1

根据银行家算法**不负荷**原则【一个进程的最大需求量不超过系统拥有的总资源数，才会被接纳执行】，优先给进程 P2 执行，因为剩余的 0 1 0 资源够让 P2 执行。

第二步：执行 P2

P2 执行之后，释放了刚刚放入的 2 1 0 资源，而且可以释放已分配的 2 1 1 资源，所以此时的资源剩余量。

资源 R1 的剩余可用资源数 = 原资源数 - 执行 P2 消耗数 + P2 执行完释放的资源数 = $2 - 0 + (2 + 0) = 4$ 。

资源 R2 的剩余可用资源数 = 原资源数 - 执行 P2 消耗数 + P2 执行完释放的资源数 = $1 - 1 + (1 + 1) = 2$ 。

资源 R3 的剩余可用资源数 = 原资源数 - 执行 P2 消耗数 + P2 执行完释放的资源数 = $0 - 0 + (0 + 1) = 1$ 。

执行完成 P2 后，操作系统剩余可用资源数为 4 2 1。

进程	最大需求量	已分配资源数	第二次分配需要的资源数
P1	6 5 2	1 2 1	5 3 1
P2	完成	完成	完成
P3	8 1 1	2 1 0	6 0 1
P4	1 2 1	1 2 0	0 0 1
P5	3 4 4	1 1 3	2 3 1

第三步：执行 P4

此时操作系统剩余可用资源数为 4 2 1，只能执行进程 P4，因为其他进程资源不够。

P4 执行之后，释放了刚刚放入的 0 0 1 资源，而且可以释放已分配的 1 2 1 资源，所以此时的资源剩余量。

资源 R1 的剩余可用资源数 = 原资源数 - 执行 P4 消耗数 + P4 执行完释放的资源数 = $4 - 0 + (1 + 0) = 5$ 。

资源 R2 的剩余可用资源数 = 原资源数 - 执行 P4 消耗数 + P4 执行完释放的资源数 = $2 - 0 + (2 + 0) = 4$ 。

资源 R3 的剩余可用资源数 = 原资源数 - 执行 P4 消耗数 + P4 执行完释放的资源数 = 1 - 1 + (0 + 1) = 1。

执行完成 P4 后，操作系统剩余可用资源数为 5 4 1。

进程	最大需求量	已分配资源数	第三次分配需要的资源数
P1	6 5 2	1 2 1	5 3 1
P2	完成	完成	完成
P3	8 1 1	2 1 0	6 0 1
P4	完成	完成	完成
P5	3 4 4	1 1 3	2 3 1

第四步：执行 P5

此时操作系统剩余可用资源数为 5 4 1，只能执行进程 P5，因为其他进程资源不够。

P5 执行之后，释放了刚刚放入的 2 3 1 资源，而且可以释放已分配的 1 1 3 资源，所以此时的资源剩余量。

资源 R1 的剩余可用资源数 = 原资源数 - 执行 P5 消耗数 + P5 执行完释放的资源数 = 5 - 2 + (1 + 2) = 6。

资源 R2 的剩余可用资源数 = 原资源数 - 执行 P5 消耗数 + P5 执行完释放的资源数 = 4 - 3 + (1 + 3) = 5。

资源 R3 的剩余可用资源数 = 原资源数 - 执行 P5 消耗数 + P5 执行完释放的资源数 = 1 - 1 + (3 + 1) = 4。

执行完成 P5 后，操作系统剩余可用资源数为 6 5 4。

进程	最大需求量	已分配资源数	第三次分配需要的资源数
P1	6 5 2	1 2 1	5 3 1
P2	完成	完成	完成
P3	8 1 1	2 1 0	6 0 1
P4	完成	完成	完成
P5	完成	完成	完成

第五步：执行 P1 或者 P3

此时操作系统剩余可用资源数为 6 5 4，可以执行 P1 或 P3。

所以安全执行顺序为 p2 => p4 => p5 => p1 => p3 或 p2 => p4 => p5 => p3 => p1。



或



银行家算法总结

银行家算法的核心思想，就是在**分配给进程资源前**，**首先判断这个进程的安全性**，也就是预执行，判断分配后是否产生死锁现象。如果系统当前资源能满足其执行，则尝试分配，如果不满足则让该进程等待。

通过不断检查剩余可用资源是否满足某个进程的最大需求，如果可以则加入安全序列，并把该进程当前持有的资源回收；不断重复这个过程，看最后能否实现让所有进程都加入安全序列。安全序列一定不会发生死锁，但没有死锁不一定是安全序列。

乐观锁和悲观锁有什么区别？

乐观锁：

- 基本思想：乐观锁假设多个事务之间很少发生冲突，因此在读取数据时不会加锁，而是在更新数据时检查数据的版本（如使用版本号或时间戳），如果版本匹配则执行更新操作，否则认为发生了冲突。
- 使用场景：乐观锁适用于读多写少的场景，可以减少锁的竞争，提高并发性能。例如，数据库中的乐观锁机制可以用于处理并发更新同一行数据的情况。

悲观锁：

- 基本思想：悲观锁假设多个事务之间会频繁发生冲突，因此在读取数据时会加锁，防止其他事务对数据进行修改，直到当前事务完成操作后才释放锁。
- 使用场景：悲观锁适用于写多的场景，通过加锁保证数据的一致性。例如，数据库中的行级锁机制可以用于处理并发更新同一行数据的情况。

乐观锁适用于读多写少的场景，通过版本控制来处理冲突；而悲观锁适用于写多的场景，通过加锁来避免冲突。

内存管理

介绍一下操作系统内存管理

操作系统设计了虚拟内存，每个进程都有自己的独立的虚拟内存，我们所写的程序不会直接与物理内存打交道。

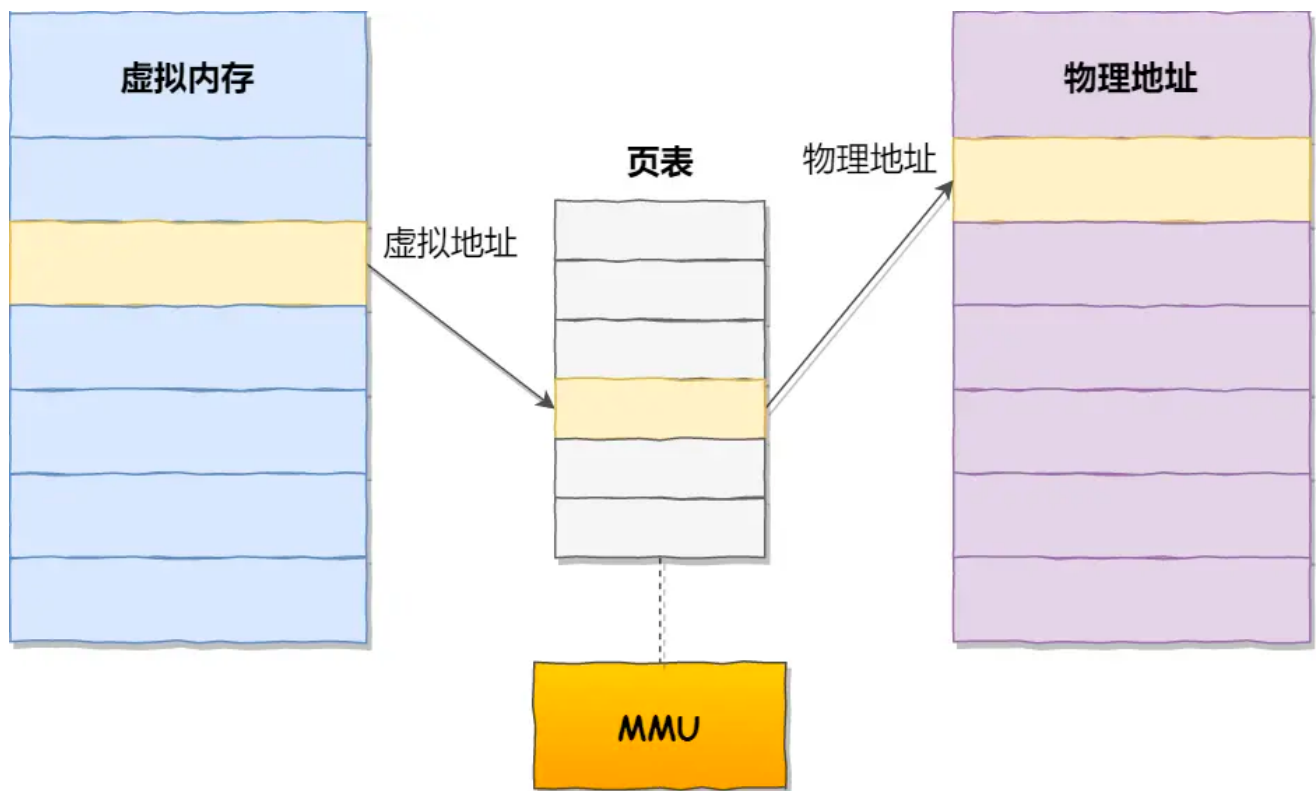


有了虚拟内存之后，它带来了这些好处：

- 第一，虚拟内存可以使得进程对运行内存超过物理内存大小，因为程序运行符合局部性原理，CPU 访问内存会有很明显的重复访问的倾向性，对于那些没有被经常使用到的内存，我们可以把它换出到物理内存之外，比如硬盘上的 swap 区域。
- 第二，由于每个进程都有自己的页表，所以每个进程的虚拟内存空间就是相互独立的。进程也没有办法访问其他进程的页表，所以这些页表是私有的，这就解决了多进程之间地址冲突的问题。
- 第三，页表里的页表项中除了物理地址之外，还有一些标记属性的比特，比如控制一个页的读写权限，标记该页是否存在等。在内存访问方面，操作系统提供了更好的安全性。

Linux 是通过对内存分页的方式来管理内存，**分页是把整个虚拟和物理内存空间切成一段段固定尺寸的大小**。这样一个连续并且尺寸固定的内存空间，我们叫**页 (Page)**。在 Linux 下，每一页的大小为 4KB。

虚拟地址与物理地址之间通过**页表**来映射，如下图：



页表是存储在内存里的，**内存管理单元**（MMU）就做将虚拟内存地址转换成物理地址的工作。

而当进程访问的虚拟地址在页表中查不到时，系统会产生一个**缺页异常**，进入系统内核空间分配物理内存、更新进程页表，最后再返回用户空间，恢复进程的运行。

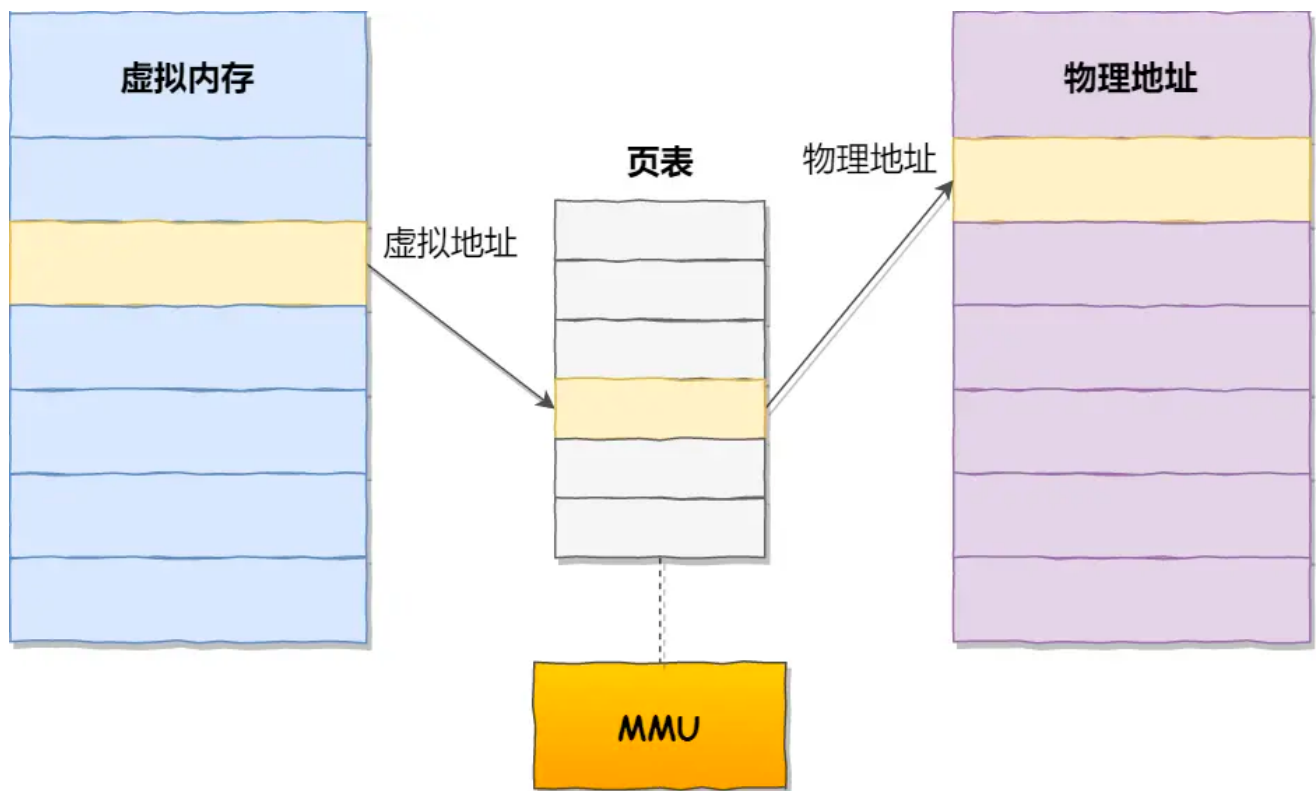
什么是虚拟内存和物理内存？

- 虚拟内存：是操作系统提供给每个运行中程序的一种地址空间，每个程序在运行时认为自己拥有的内存空间就是虚拟内存，其大小可以远远大于物理内存的大小。虚拟内存通过将程序的地址空间划分成若干个固定大小的页或段，并将这些页或者段映射到物理内存中的不同位置，从而使得程序在运行时可以更高效地利用物理内存。
- 物理内存：物理内存是计算机实际存在的内存，是计算机中的实际硬件部件。

讲一下页表？

分页是把整个虚拟和物理内存空间切成一段段固定尺寸的大小。这样一个连续并且尺寸固定的内存空间，我们叫**页**（Page）。在 Linux 下，每一页的大小为 4KB。

虚拟地址与物理地址之间通过**页表**来映射，如下图：



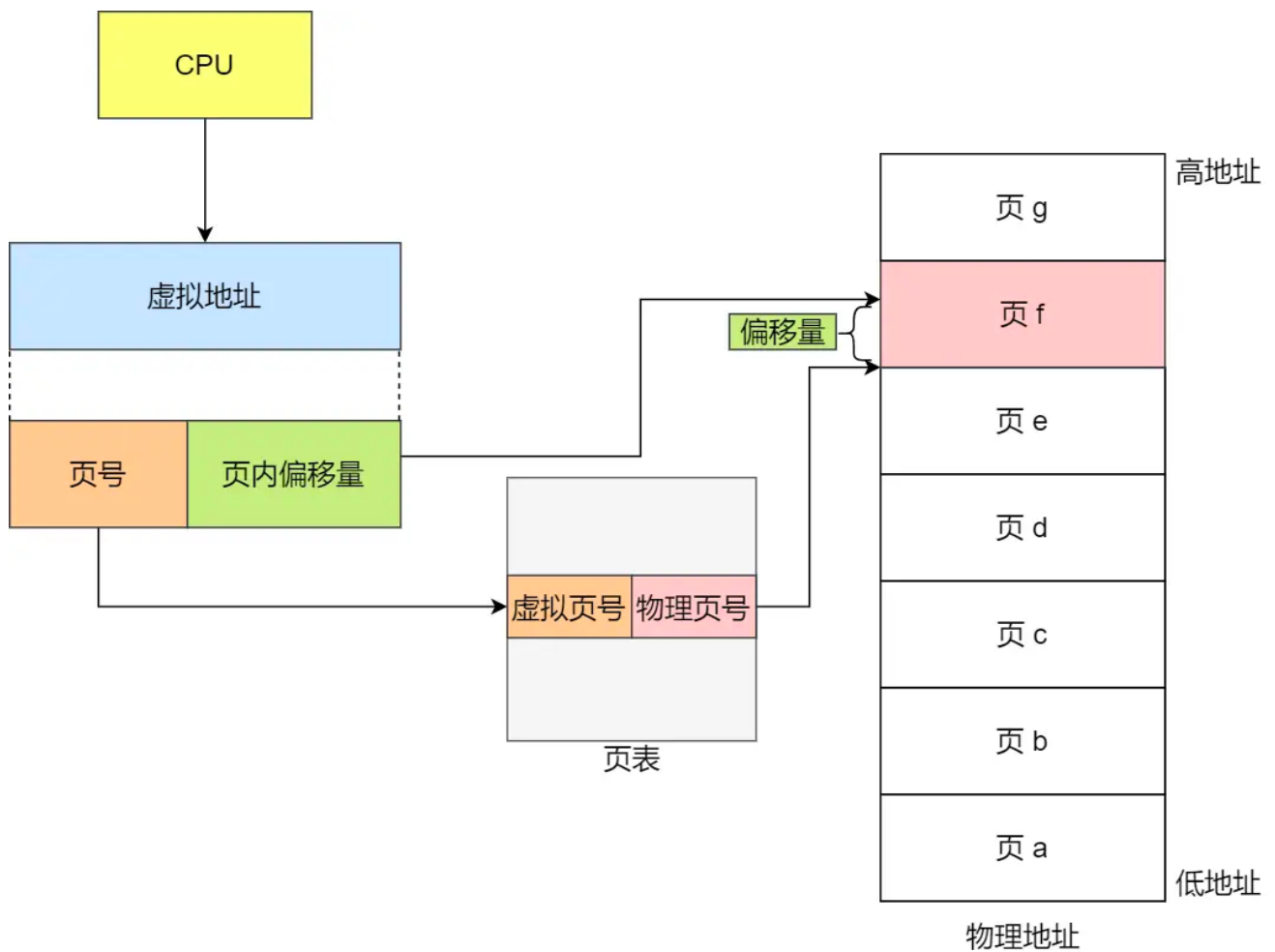
页表是存储在内存里的，**内存管理单元**（MMU）就做将虚拟内存地址转换成物理地址的工作。

而当进程访问的虚拟地址在页表中查不到时，系统会产生一个**缺页异常**，进入系统内核空间分配物理内存、更新进程页表，最后再返回用户空间，恢复进程的运行。

内存分页由于内存空间都是预先划分好的，也就不会像内存分段一样，在段与段之间会产生间隙非常小的内存，这正是分段会产生外部内存碎片的原因。而**采用了分页，页与页之间是紧密排列的，所以不会有外部碎片**。

但是，因为内存分页机制分配内存的最小单位是一页，即使程序不足一页大小，我们最少只能分配一个页，所以页内会出现内存浪费，所以针对**内存分页机制会有内部内存碎片**的现象。

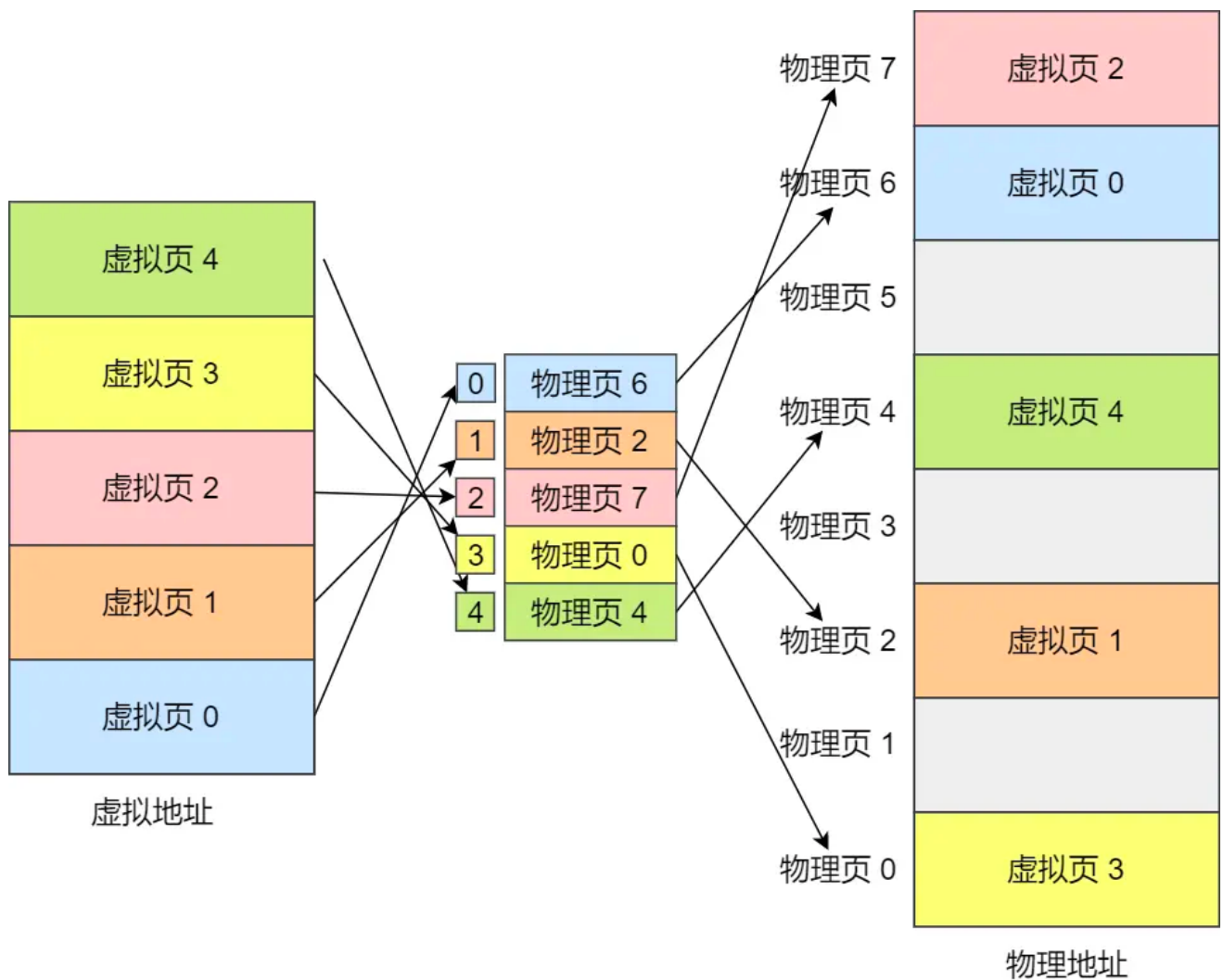
在分页机制下，虚拟地址分为两部分，**页号**和**页内偏移**。页号作为页表的索引，**页表**包含物理页每页所在**物理内存的基地址**，这个基地址与页内偏移的组合就形成了物理内存地址，见下图。



总结一下，对于一个内存地址转换，其实就是这样三个步骤：

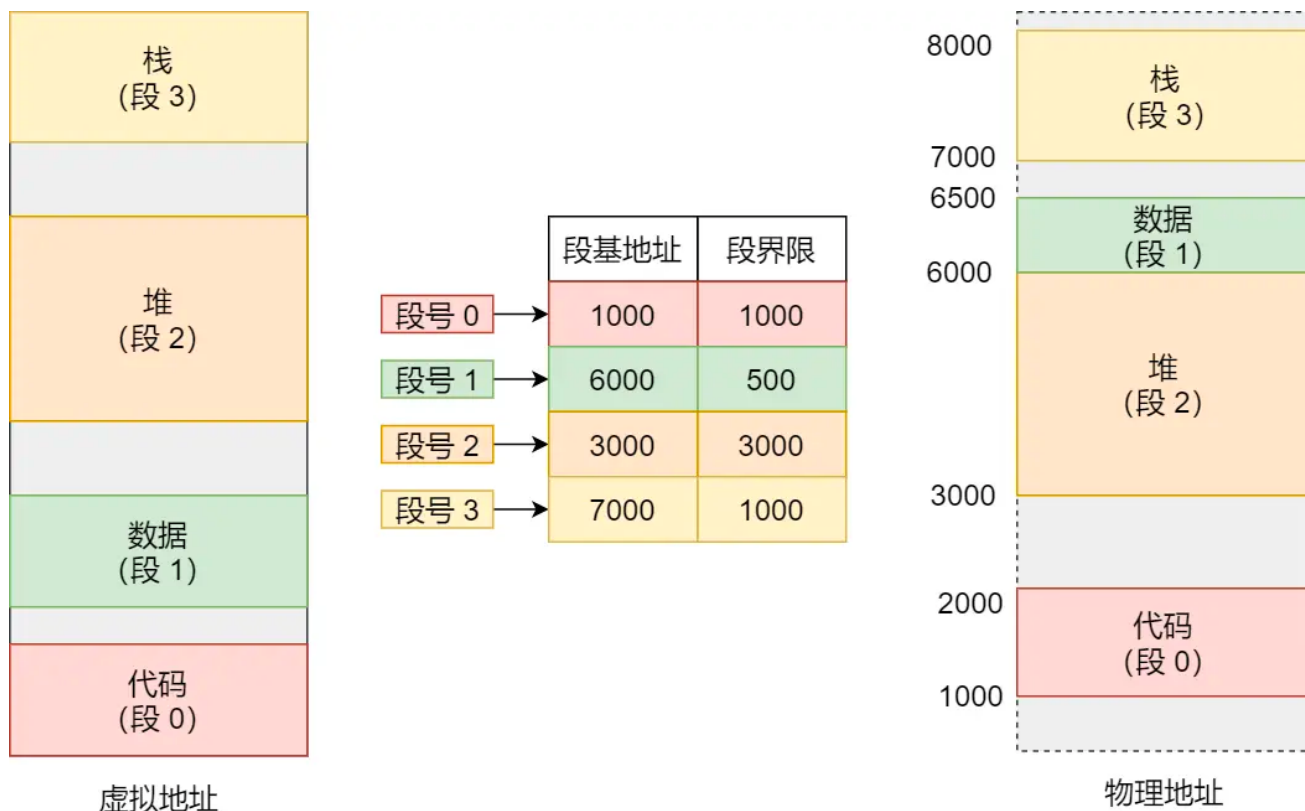
- 把虚拟内存地址，切分成页号和偏移量；
- 根据页号，从页表里面，查询对应的物理页号；
- 直接拿物理页号，加上前面的偏移量，就得到了物理内存地址。

下面举个例子，虚拟内存中的页通过页表映射为了物理内存中的页，如下图：



讲一下段表?

虚拟地址也可以通过**段表**与物理地址进行映射的，分段机制会把程序的虚拟地址分成 4 个段，每个段在段表中有一个项，在这一项找到段的基地址，再加上偏移量，于是就能找到物理内存中的地址，如下图：



如果要访问段 3 中偏移量 500 的虚拟地址，我们可以计算出物理地址为，段 3 基地址 7000 + 偏移量 500 = 7500。

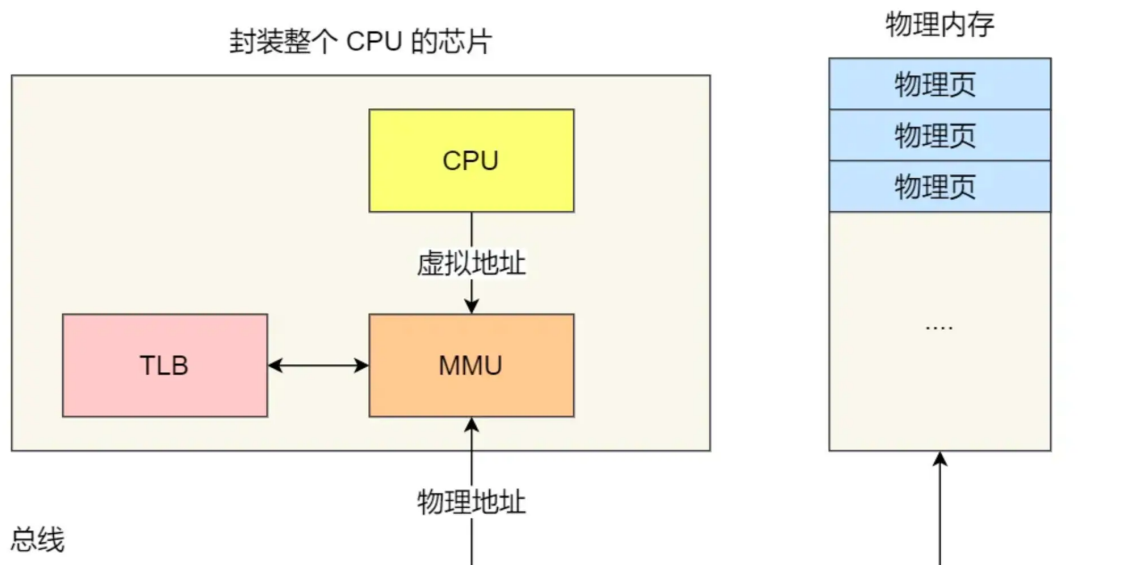
虚拟地址是怎么转化到物理地址的？

虚拟地址转化为物理地址是通过内存管理单元（Memory Management Unit, MMU）来完成的。MMU是计算机系统硬件组件，负责虚拟地址和物理地址之间的转换。

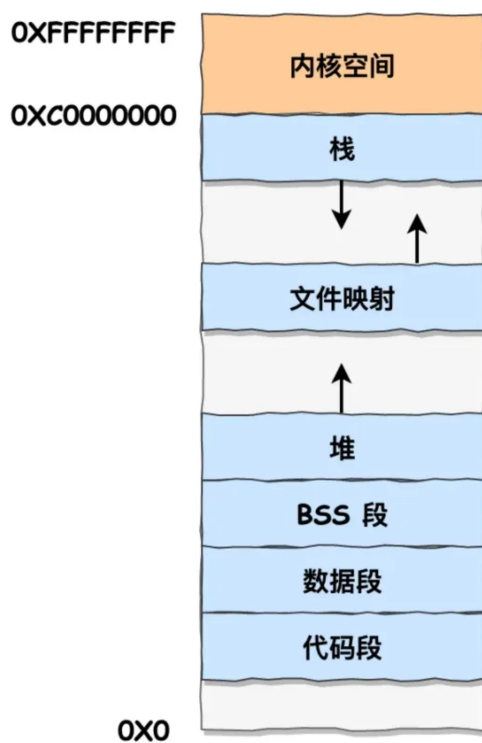
在虚拟地址转换的过程中，通常会使用页表（Page Table）来进行映射。页表是一种数据结构，它将虚拟地址空间划分为固定大小的页（Page），对应于物理内存中的页框（Page Frame）。每个页表项（Page Table Entry）记录了虚拟页和物理页的对应关系。

当程序访问一个虚拟地址时，MMU会将虚拟地址分解为页号和页内偏移量。然后，MMU会查找页表，根据页号找到对应的页表项。页表项中包含了物理页的地址或页框号。最后，MMU将物理页的地址与页内偏移量组合，得到对应的物理地址。

虚拟地址转化为物理地址的过程中，还可能涉及到多级页表、TLB（Translation Lookaside Buffer）缓存等机制，以提高地址转换的效率。



程序的内存布局是怎样的？



通过这张图你可以看到，用户空间内存，从**低到高**分别是 6 种不同的内存段：

- 代码段，包括二进制可执行代码；
- 数据段，包括已初始化的静态常量和全局变量；
- BSS 段，包括未初始化的静态变量和全局变量；
- 堆段，包括动态分配的内存，从低地址开始向上增长；
- 文件映射段，包括动态库、共享内存等；
- 栈段，包括局部变量和函数调用的上下文等。栈的大小是固定的，一般是 8 MB。当然系统也提供了参数，以便我们自定义大小；

上图中的内存布局可以看到，代码段下面还有一段内存空间的（灰色部分），这一块区域是「保留区」，之所以要有保留区这是因为在大多数的系统里，我们认为比较小数值的地址不是一个合法地址，例如，我们通常在 C 的代码里会将无效的指针赋值为 NULL。因此，这里会出现一段不可访问的内存保留区，防止程序因为出现 bug，导致读或写了一些小内存地址的数据，而使得程序跑飞。

在这 7 个内存段中，堆和文件映射段的内存是动态分配的。比如说，使用 C 标准库的 `malloc()` 或者 `mmap()`，就可以分别在堆和文件映射段动态分配内存。

堆和栈的区别？

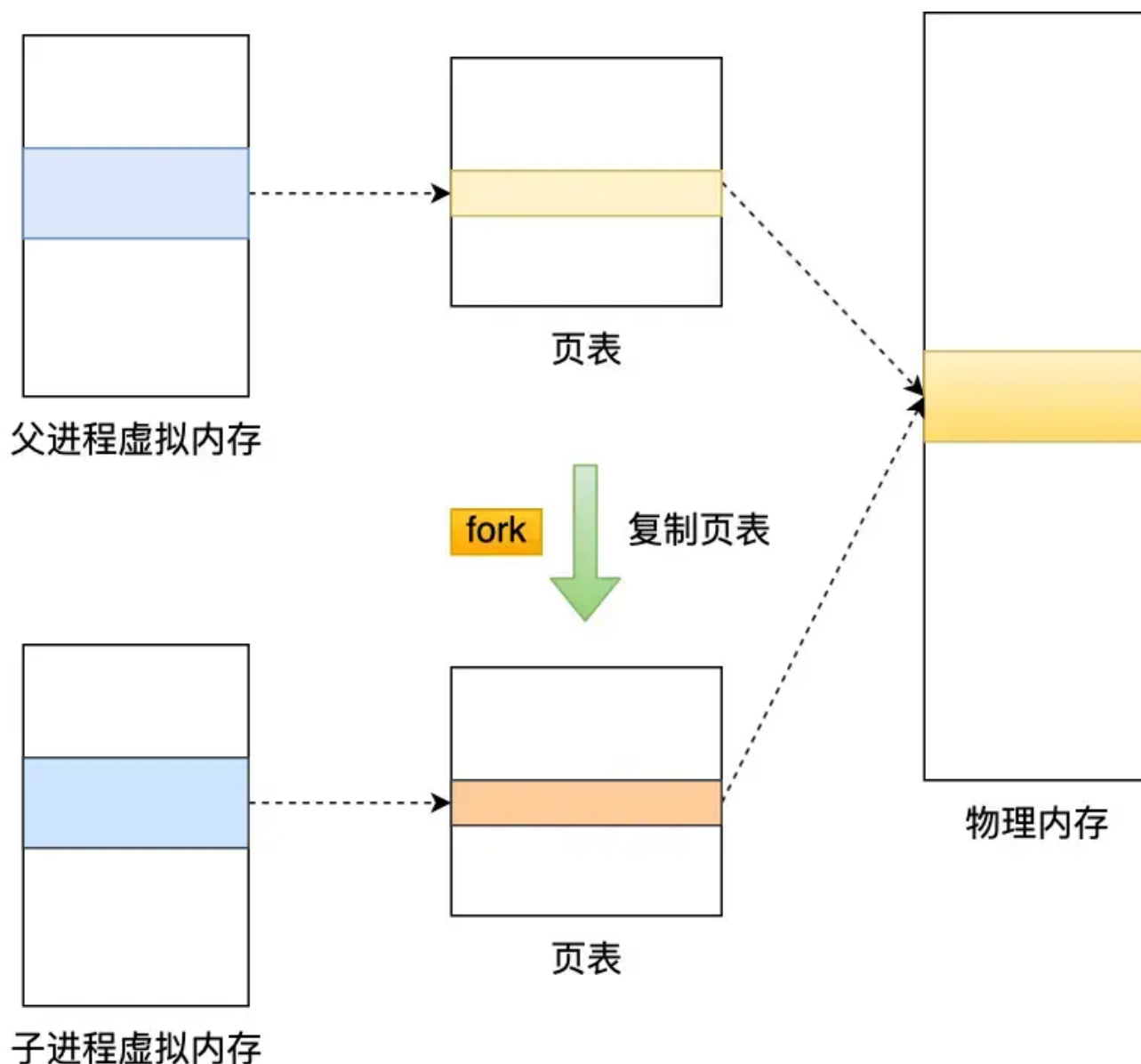
- **分配方式**：堆是动态分配内存，由程序员手动申请和释放内存，通常用于存储动态数据结构和对象。栈是静态分配内存，由编译器自动分配和释放内存，用于存储函数的局部变量和函数调用信息。
- **内存管理**：堆需要程序员手动管理内存的分配和释放，如果管理不当可能会导致内存泄漏或内存溢出。栈由编译器自动管理内存，遵循后进先出的原则，变量的生命周期由其作用域决定，函数调用时分配内存，函数返回时释放内存。
- **大小和速度**：堆通常比栈大，内存空间较大，动态分配和释放内存需要时间开销。栈大小有限，通常比较小，内存分配和释放速度较快，因为是编译器自动管理。

fork()会复制哪些东西？

- fork 阶段会复制父进程的页表（虚拟内存）
- fork 之后，如果发生了写时复制，就会复制物理内存

介绍copy on write(写时复制)

主进程在执行 fork 的时候，操作系统会把主进程的「页表」复制一份给子进程，这个页表记录着虚拟地址和物理地址映射关系，而不会复制物理内存，也就是说，两者的虚拟空间不同，但其对应的物理空间是同一个。



这样一来，子进程就共享了父进程的物理内存数据了，这样能够**节约物理内存资源**，页表对应的页表项的属性会标记该物理内存的权限为**只读**。

不过，当父进程或者子进程在向这个内存发起写操作时，CPU 就会触发**写保护中断**，这个写保护中断是由于违反权限导致的，然后操作系统会在「写保护中断处理函数」里进行**物理内存的复制**，并重新设置其内存映射关系，将父子进程的内存读写权限设置为**可读写**，最后才会对内存进行写操作，这个过程被称为「**写时复制(Copy On Write)**」。

写时复制顾名思义，**在发生写操作的时候，操作系统才会去复制物理内存**，这样是为了防止 `fork` 创建子进程时，由于物理内存数据的复制时间过长而导致父进程长时间阻塞的问题。

copy on write节省了什么资源？

节省了物理内存的资源，因为 `fork` 的时候，子进程不需要复制父进程的物理内存，避免了不必要的内存复制开销，子进程只需要复制父进程的页表，这时候父子进程的页表指向的都是共享的物理内存。

只有当父子进程任何一方对这片共享的物理内存发生了修改操作，才会触发写时复制机制，这时候才会复制发生修改操作的物理内存。

malloc 1KB和1MB 有什么区别？

malloc() 源码里默认定义了一个阈值：

- 如果用户分配的内存小于 128 KB，则通过 brk() 申请内存；
- 如果用户分配的内存大于 128 KB，则通过 mmap() 申请内存；

注意，不同的 glibc 版本定义的阈值也是不同的。

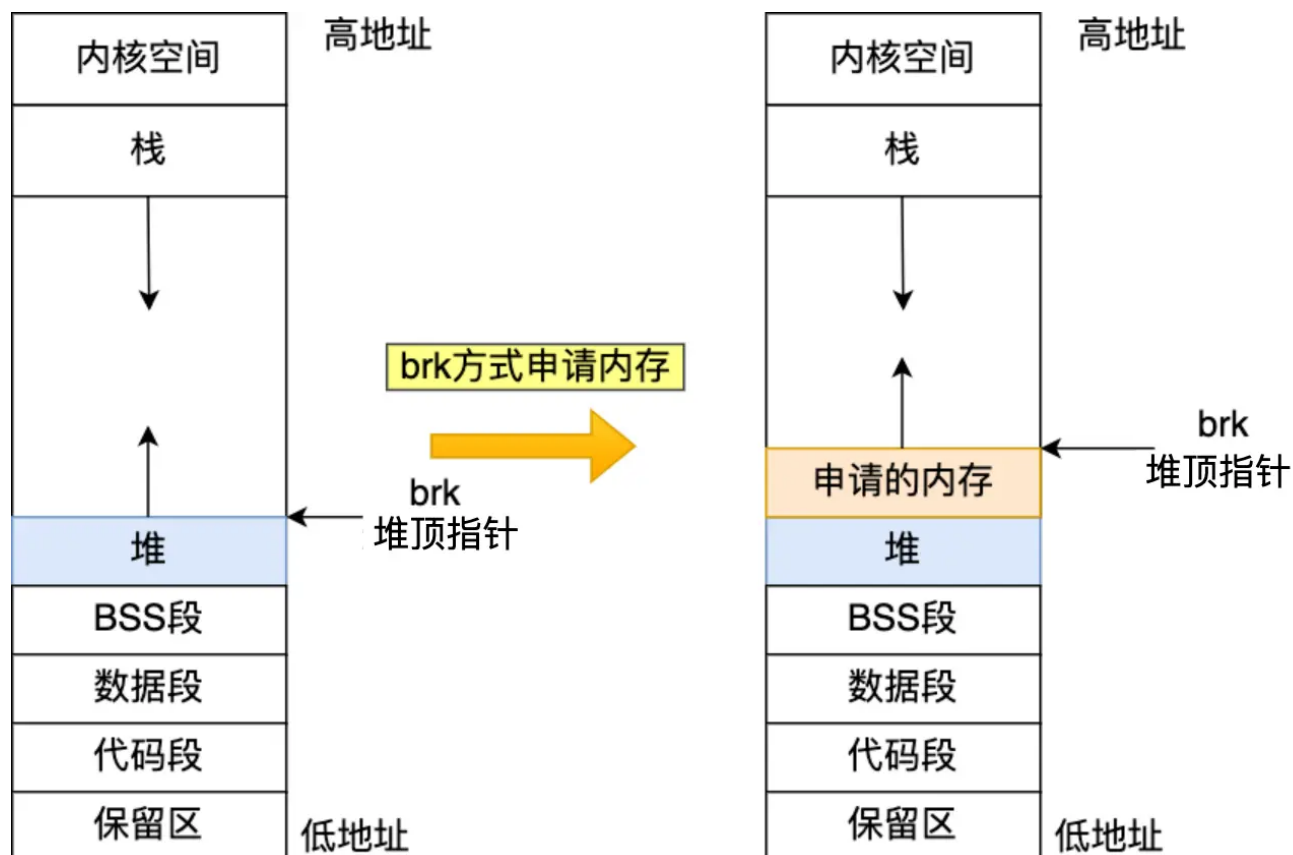
介绍一下brk，mmap

实际上，malloc() 并不是系统调用，而是 C 库里的函数，用于动态分配内存。

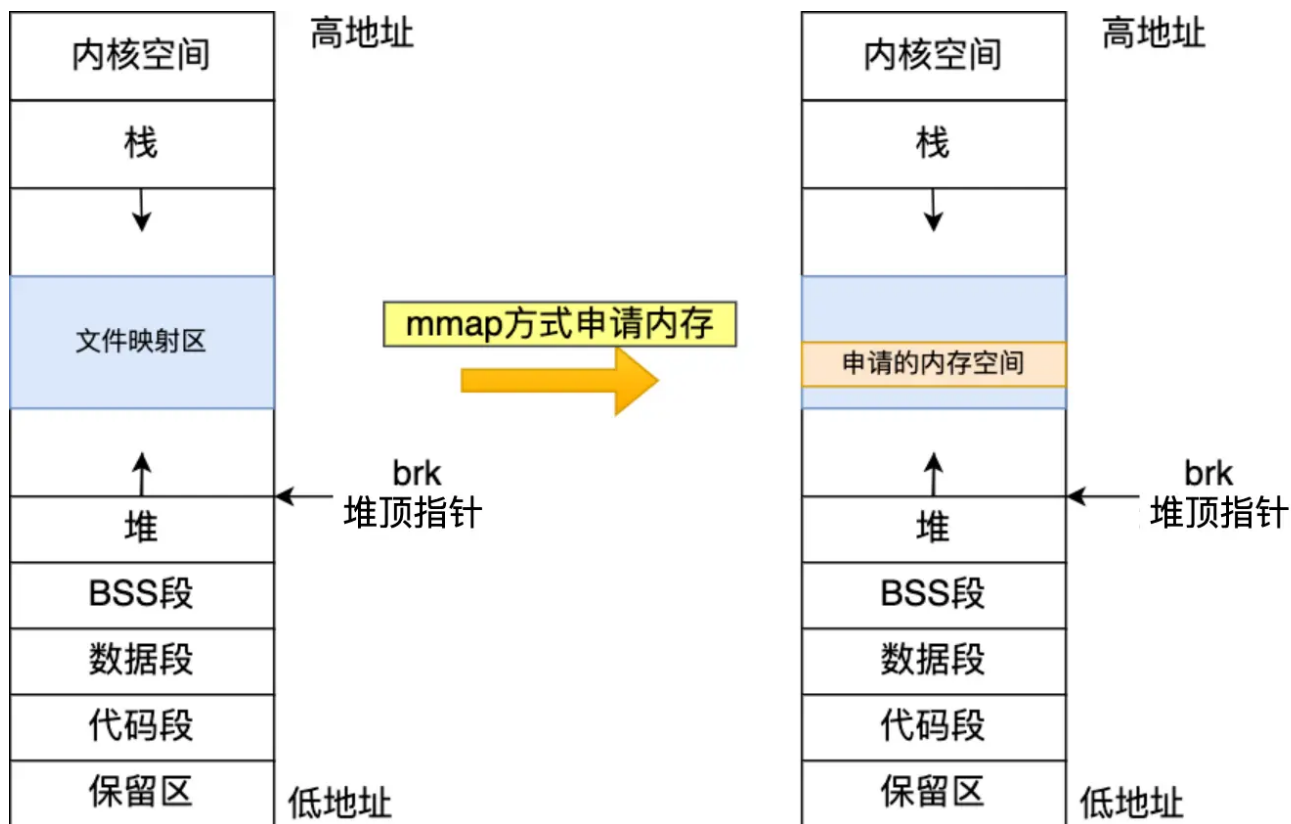
malloc 申请内存的时候，会有两种方式向操作系统申请堆内存。

- 方式一：通过 brk() 系统调用从堆分配内存
- 方式二：通过 mmap() 系统调用在文件映射区域分配内存；

方式一实现的方式很简单，就是通过 brk() 函数将「堆顶」指针向高地址移动，获得新的内存空间。如下图：



方式二通过 `mmap()` 系统调用中「私有匿名映射」的方式，在文件映射区分配一块内存，也就是从文件映射区“偷”了一块内存。如下图：



操作系统内存不足时会发生什么？

应用程序通过 malloc 函数申请内存的时候，实际上申请的是虚拟内存，此时并不会分配物理内存。

当应用程序读写了这块虚拟内存，CPU 就会去访问这个虚拟内存，这时会发现这个虚拟内存没有映射到物理内存，CPU 就会产生**缺页中断**，进程会从用户态切换到内核态，并将缺页中断交给内核的 Page Fault Handler（缺页中断函数）处理。

缺页中断处理函数会看是否有空闲的物理内存，如果有，就直接分配物理内存，并建立虚拟内存与物理内存之间的映射关系。

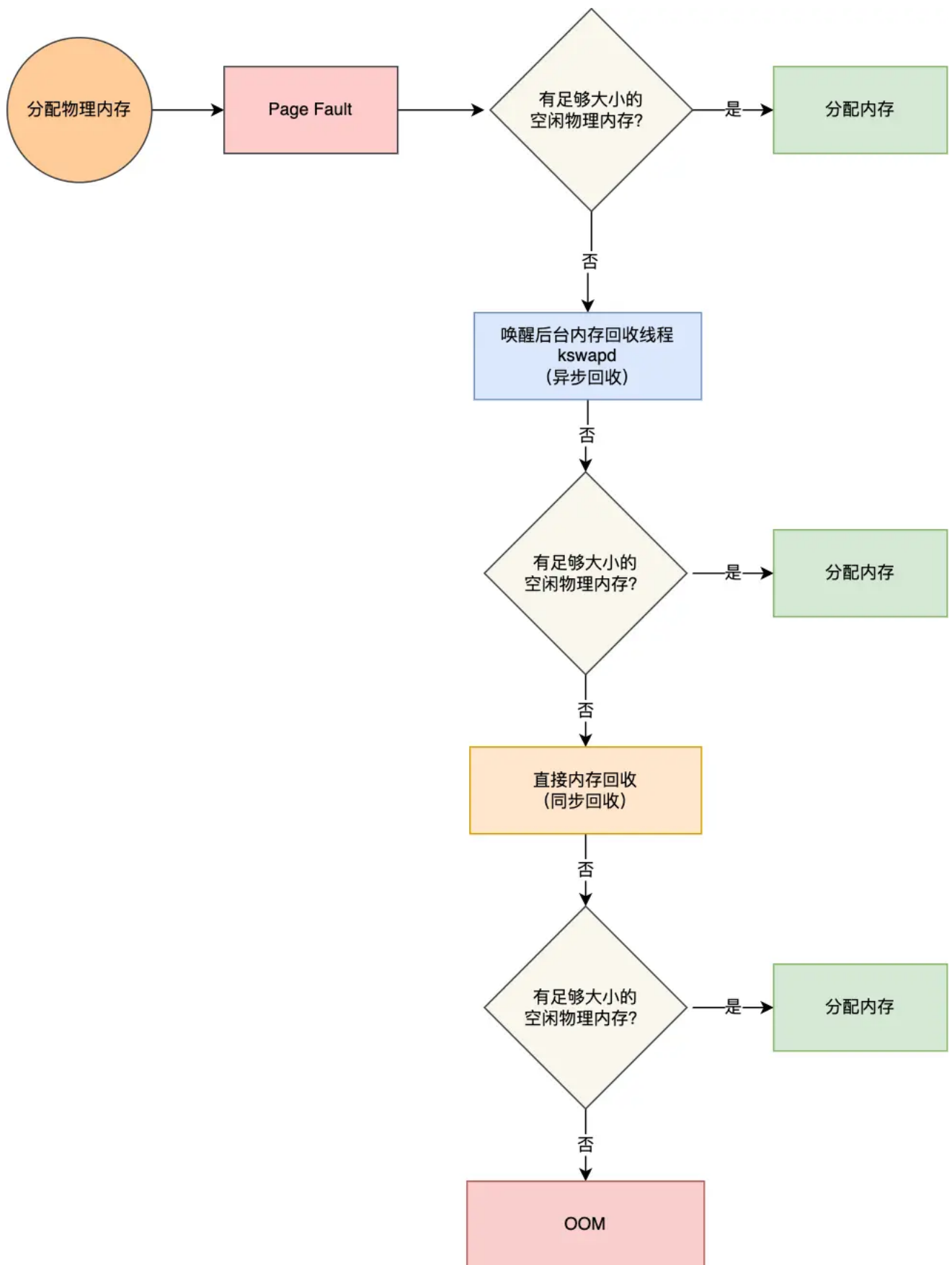
如果没有空闲的物理内存，那么内核就会开始进行**回收内存**的工作，回收的方式主要是两种：直接内存回收和后台内存回收。

- **后台内存回收** (kswapd)：在物理内存紧张的时候，会唤醒 kswapd 内核线程来回收内存，这个回收内存的过程**异步**的，不会阻塞进程的执行。
- **直接内存回收** (direct reclaim)：如果后台异步回收跟不上进程内存申请的速度，就会开始直接回收，这个回收内存的过程是**同步**的，会阻塞进程的执行。

如果直接内存回收后，空闲的物理内存仍然无法满足此次物理内存的申请，那么内核就会放最后的大招了——**触发 OOM (Out of Memory) 机制**。

OOM Killer 机制会根据算法选择一个占用物理内存较高的进程，然后将其杀死，以便释放内存资源，如果物理内存依然不足，OOM Killer 会继续杀死占用物理内存较高的进程，直到释放足够的内存位置。

申请物理内存的过程如下图：



系统内存紧张的时候，就会进行回收内存的工作，那具体哪些内存是可以被回收的呢？

主要有两类内存可以被回收，而且它们的回收方式也不同。

- **文件页** (File-backed Page)：内核缓存的磁盘数据 (Buffer) 和内核缓存的文件数据 (Cache) 都叫作文件页。大部分文件页，都可以直接释放内存，以后有需要时，再从磁盘重新读取就可以了。而那些被应用程序修改过，并且暂时还没写入磁盘的数据（也就是脏页），就得先写入磁盘，然后才能进行内存释放。所以，**回收干净页的方式是直接释放内存，回收脏页的方式是先写回磁盘后再释放内存。**
- **匿名页** (Anonymous Page)：这部分内存没有实际载体，不像文件缓存有硬盘文件这样一个载体，比如堆、栈数据等。这部分内存很可能还要再次被访问，所以不能直接释放内存，它们**回收的方式是通过 Linux 的 Swap 机制**，Swap 会把不常访问的内存先写到磁盘中，然后释放这些内存，给其他更需要的进程使用。再次访问这些内存时，重新从磁盘读入内存就可以了。

文件页和匿名页的回收都是**基于 LRU 算法**，也就是优先回收不常访问的内存。LRU 回收算法，实际上维护着 active 和 inactive 两个双向链表，其中：

- **active_list** 活跃内存页链表，这里存放的是最近被访问过（活跃）的内存页；
- **inactive_list** 不活跃内存页链表，这里存放的是很少被访问（非活跃）的内存页；

越接近链表尾部，就表示内存页越不常访问。这样，在回收内存时，系统就可以根据活跃程度，优先回收不活跃的内存。

页面置换有哪些算法？

页面置换算法的功能是，**当出现缺页异常，需调入新页面而内存已满时，选择被置换的物理页面**，也就是说选择一个物理页面换出到磁盘，然后把需要访问的页面换入到物理页。

那其算法目标则是，尽可能减少页面的换入换出的次数，常见的页面置换算法有如下几种：

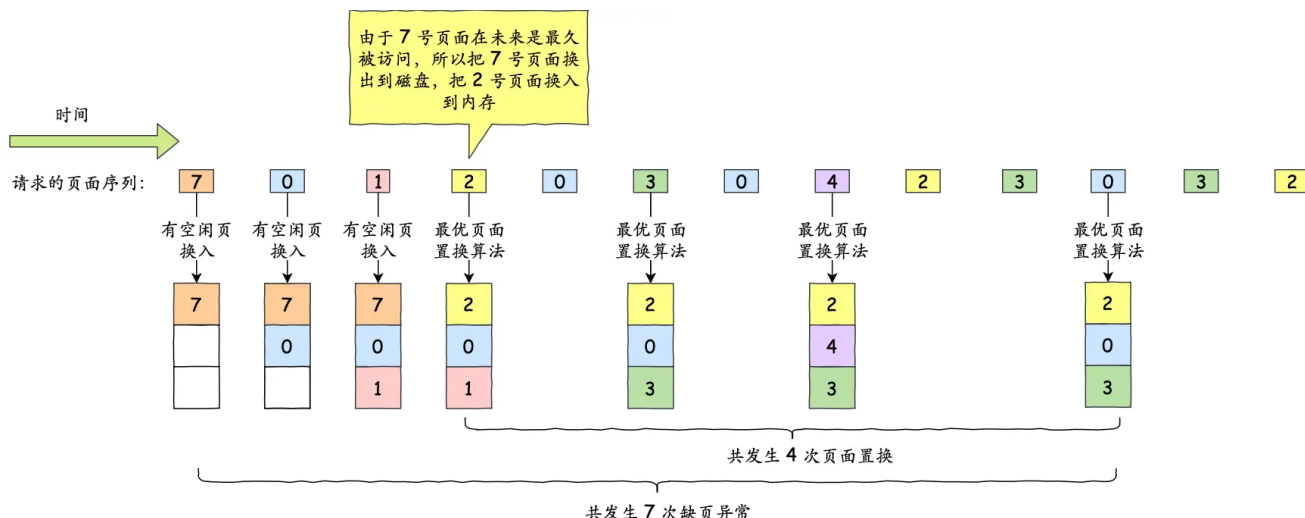
- 最佳页面置换算法 (OPT)
- 先进先出置换算法 (FIFO)
- 最近最久未使用的置换算法 (LRU)
- 时钟页面置换算法 (Clock)
- 最不常用置换算法 (LFU)

最佳页面置换算法

最佳页面置换算法基本思路是，**置换在「未来」最长时间不访问的页面。**

所以，该算法实现需要计算内存中每个逻辑页面的「下一次」访问时间，然后比较，选择未来最长时间不访问的页面。

我们举个例子，假设一开始有 3 个空闲的物理页，然后有请求的页面序列，那它的置换过程如下图：



在这个请求的页面序列中，缺页共发生了 7 次（空闲页换入 3 次 + 最优页面置换 4 次），页面置换共发生了 4 次。

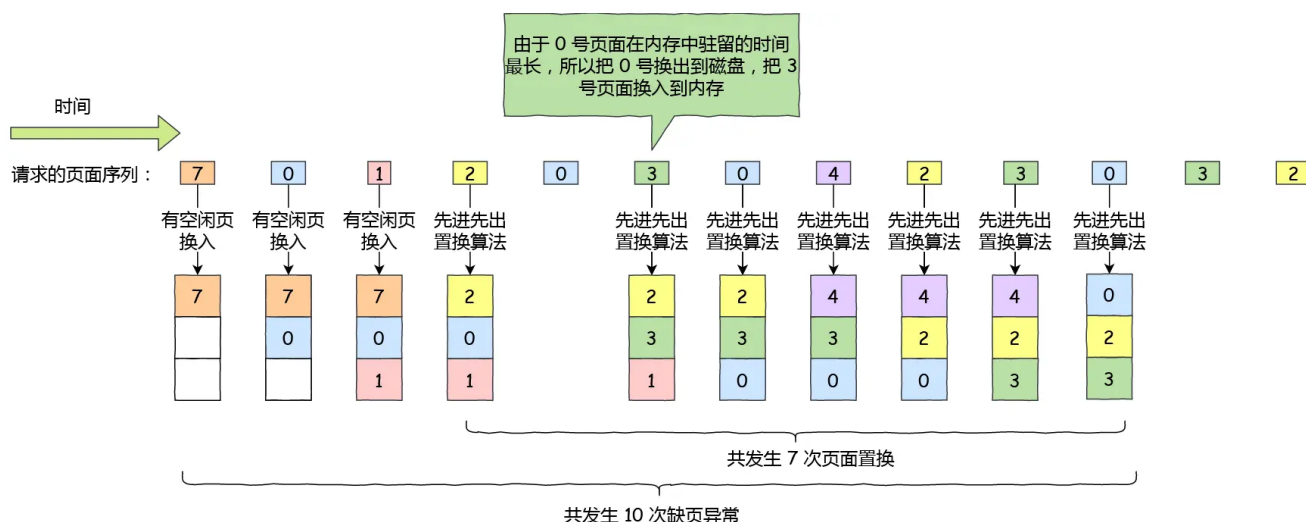
这很理想，但是实际系统中无法实现，因为程序访问页面时是动态的，我们是无法预知每个页面在「下一次」访问前的等待时间。

所以，最佳页面置换算法作用是为了衡量你的算法的效率，你的算法效率越接近该算法的效率，那么说明你的算法是高效的。

先进先出置换算法

既然我们无法预知页面在下次访问前所需的等待时间，那我们可以选择在内存驻留时间很长的页面进行中置换，这个就是「先进先出置换」算法的思想。

还是以前面的请求的页面序列作为例子，假设使用先进先出置换算法，则过程如下图：



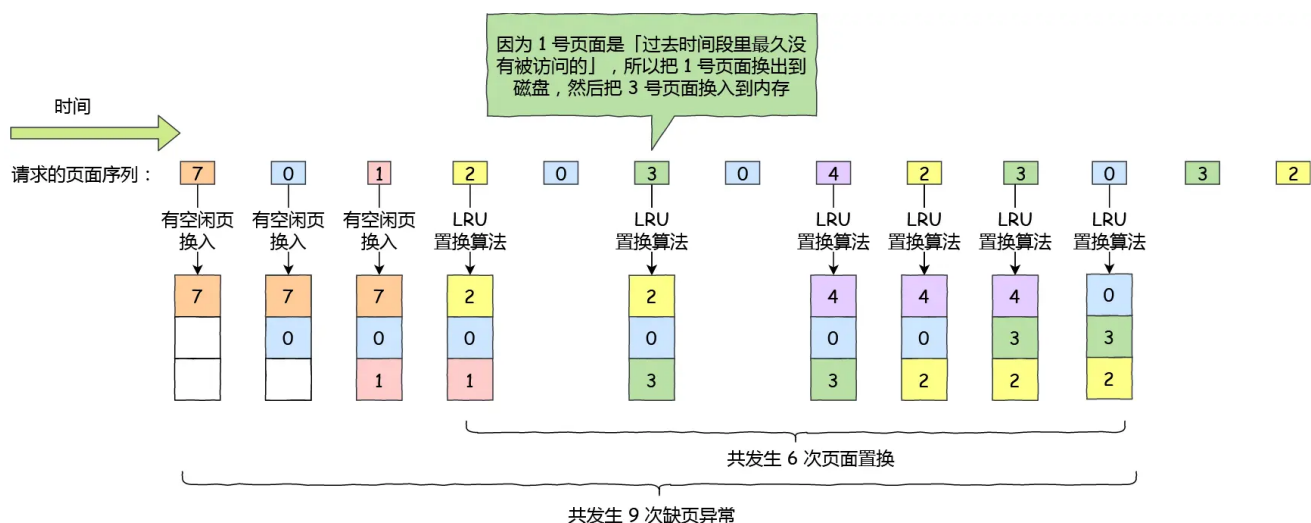
在这个请求的页面序列中，缺页共发生了 10 次，页面置换共发生了 7 次，跟最佳页面置换算法比较起来，性能明显差了很多。

最近最久未使用的置换算法

最近最久未使用（LRU）的置换算法的基本思路是，发生缺页时，选择最长时间没有被访问的页面进行置换，也就是说，该算法假设已经很久没有使用的页面很有可能在未来较长的一段时间内仍然不会被使用。

这种算法近似最优置换算法，最优置换算法是通过「未来」的使用情况来推测要淘汰的页面，而 LRU 则是通过「历史」的使用情况来推测要淘汰的页面。

还是以前面的请求的页面序列作为例子，假设使用最近最久未使用的置换算法，则过程如下图：



在这个请求的页面序列中，缺页共发生了 9 次，页面置换共发生了 6 次，跟先进先出置换算法比较起来，性能提高了一些。

虽然 LRU 在理论上是可以实现，但代价很高。为了完全实现 LRU，需要在内存中维护一个所有页面的链表，最近最多使用的页面在表头，最近最少使用的页面在表尾。

困难的是，在每次访问内存时都必须更新「整个链表」。在链表中找到一个页面，删除它，然后把它移动到表头是一个非常费时的操作。

所以，LRU 虽然看上去不错，但是由于开销比较大，实际应用中比较少使用。

时钟页面置换算法

那有没有一种即能优化置换的次数，也能方便实现的算法呢？

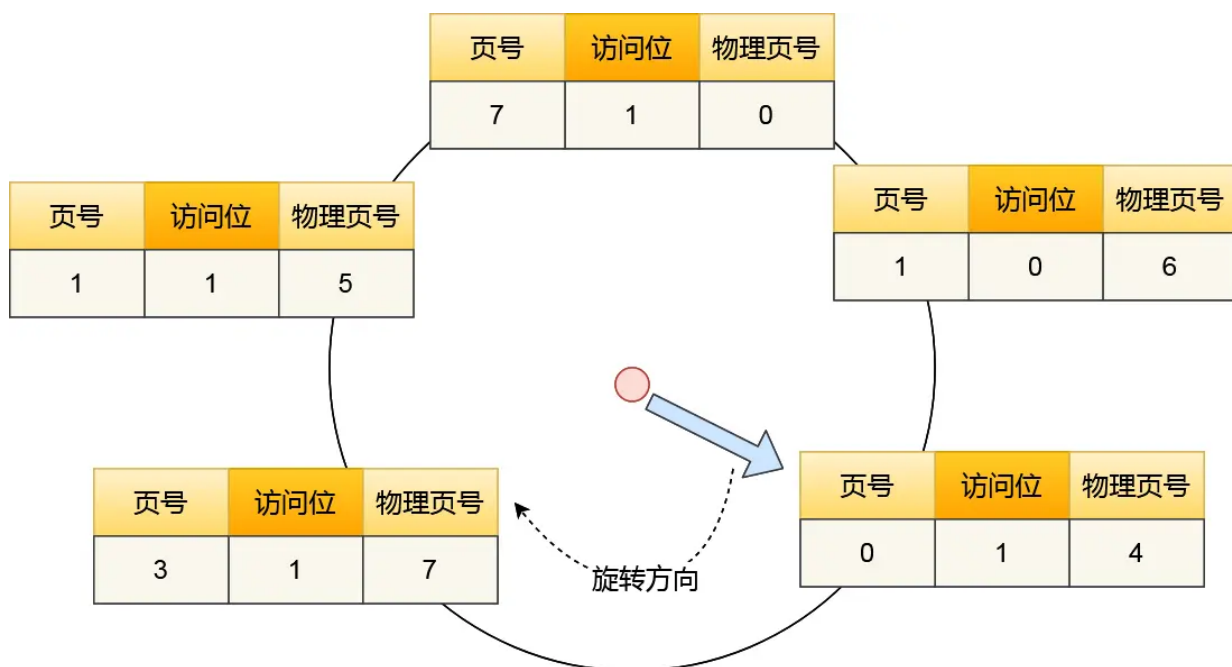
时钟页面置换算法就可以两者兼得，它跟 LRU 近似，又是对 FIFO 的一种改进。

该算法的思路是，把所有的页面都保存在一个类似钟面的「环形链表」中，一个表针指向最老的页面。

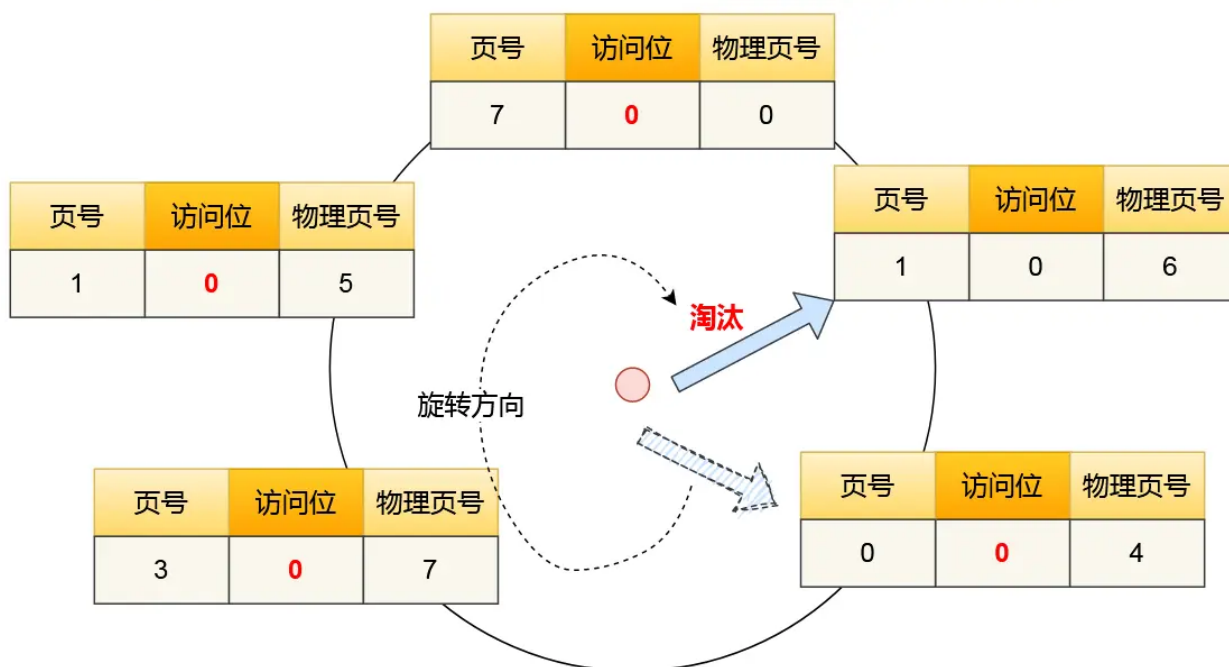
当发生缺页中断时，算法首先检查表针指向的页面：

- 如果它的访问位是 0 就淘汰该页面，并把新的页面插入这个位置，然后把表针前移一个位置；
- 如果访问位是 1 就清除访问位，并把表针前移一个位置，重复这个过程直到找到了一个访问位为 0 的页面为止；

我画了一副时钟页面置换算法的工作流程图，你可以在下方看到：



当发生缺页中断后
会把沿途遇到的访问位为 1 的修改为 0 ,
直到遇到访问位为 0 的页面 , 并将其淘汰.



了解了这个算法的工作方式，就明白为什么它被称为时钟（Clock）算法了。

最不常用算法

最不常用（LFU）算法，这个名字听起来很调皮，但是它的意思不是指这个算法不常用，而是**当发生缺页中断时，选择「访问次数」最少的那个页面，并将其淘汰。**

它的实现方式是，对每个页面设置一个「访问计数器」，每当一个页面被访问时，该页面的访问计数器就累加 1。在发生缺页中断时，淘汰计数器值最小的那个页面。

看起来很简单，每个页面加一个计数器就可以实现了，但是在操作系统中实现的时候，我们需要考虑效率和硬件成本的。

要增加一个计数器来实现，这个硬件成本是比较高的，另外如果要对这个计数器查找哪个页面访问次数最小，查找链表本身，如果链表长度很大，是非常耗时的，效率不高。

但还有个问题，LFU 算法只考虑了频率问题，没考虑时间的问题，比如有些页面在过去时间里访问的频率很高，但是现在已经没有访问了，而当前频繁访问的页面由于没有这些页面访问的次数高，在发生缺页中断时，就会可能会误伤当前刚开始频繁访问，但访问次数还不高的页面。

那这个问题的解决的办法还是有的，可以定期减少访问的次数，比如当发生时间中断时，把过去时间访问的页面的访问次数除以 2，也就是说，随着时间的流失，以前的高访问次数的页面会慢慢减少，相当于加大了被置换的概率。

中断

什么是中断？

CPU停下当前的工作任务，去处理其他事情，处理完后回来继续执行刚才的任务，这一过程便是中断。

中断分为外部中断和内部中断：

- 外部中断分为可屏蔽中断和不可屏蔽中断：
 - **可屏蔽中断：通过INTR线向CPU请求的中断**，主要来自外部设备如硬盘，打印机，网卡等。此类中断并不会影响系统运行，可随时处理，甚至不处理，所以名为可屏蔽。
 - **不可屏蔽中断：通过NMI线向CPU请求的中断**，如电源掉电，硬件线路故障等。这里不可屏蔽的意思不是不可以屏蔽，不建议屏蔽，而是问题太大，屏蔽不了，不能屏蔽的意思。注：INTR和NMI都是CPU的引脚
- 内部中断分为陷阱、故障、终止：
 - **陷阱：是一种有意的，预先安排的异常事件**，一般是在编写程序时故意设下的陷阱指令，而后执行到陷阱指令后，CPU将会调用特定程序进行相应的处理，**处理结束后返回到陷阱指令的下一条指令**。如系统调用，程序调试功能等。如printf函数，最底层的实现中会有一条int 0x80指令，这就是一条陷阱指令，使用0x80号中断进行系统调用。
 - **故障：故障是在引起故障的指令被执行，但还没有执行结束时，CPU检测到的一类的意外事件**。出错时交由故障处理程序处理，**如果能处理修正这个错误，就将控制返回到引起故障的指令即CPU重新执这条指令。如果不能处理就报错**。常见的故障为缺页，当CPU引用的虚拟地址对应的物理页不存在时就会发生故障。缺页异常是能够修正的，有着专门的缺页处理程序，它会将缺失的物理页从磁盘中重新调进主存。而后再次执行引起故障的指令时便能够顺利执行了。
 - **终止：执行指令的过程中发生了致命错误，不可修复，程序无法继续运行，只能终止，通常会是一些硬件的错误**。终止处理程序不会将控制返回给原程序，而是直接终止原程序

讲讲中断的流程

中断是计算机系统中一种机制，用于在处理器执行指令时暂停当前任务，并转而执行其他任务或处理特定事件。以下是中断的基本流程：

- **发生中断**：当外部设备或者软件程序需要处理器的注意或者响应时，会发出中断信号。处理器在接收到中断信号后，会停止当前执行的指令，保存当前执行现场，并跳转到中断处理程序执行。
- **中断响应**：处理器接收到中断信号后，会根据中断向量表找到对应的中断处理程序的入口地址。处理器会保存当前执行现场（如程序计数器、寄存器状态等），以便在中断处理完成后能够恢复执行。
- **中断处理**：处理器跳转到中断处理程序的入口地址开始执行中断处理程序。中断处理程序会根据中断类型进行相应的处理，可能涉及到保存现场、处理中断事件、执行特定任务等。

中断的类型有哪些？

中断按事件来源分类，可以分为外部中断和内部中断。中断事件来自于CPU外部的被称为外部中断，来自于CPU内部的则为内部中断。

进一步细分，外部中断还可分为可屏蔽中断（maskable interrupt）和不可屏蔽中断（non-maskable interrupt）两种，而内部中断按事件是否正常来划分可分为软中断和异常两种。

- **外部中断**的中断事件来源于CPU外部，必然是某个硬件产生的，所以外部中断又被称为硬件中断（hardware interrupt）。计算机的外部设备，如网卡、声卡、显卡等都能产生中断。外部设备的中断信号是通过两根信号线通知CPU的，一根是INTR，另一根是NMI。CPU从INTR收到的中断信号都是不影响系统运行的，CPU可以选择屏蔽（通过设置中断屏蔽寄存器中的IF位），而从NMI中收到的中断信号则是影响系统运行的严重错误，不可屏蔽，因为屏蔽的意义不大，系统已经无法运行。
- **内部中断**来自于处理器内部，其中软中断是由软件主动发起的中断，常被用于系统调用（system call）；而异常则是指令执行期间CPU内部产生的错误引起的。异常也和不可屏蔽中断一样不受eflags寄存器的IF位影响，区别在于不可屏蔽中断发生的事件会导致处理器无法运行（如断电、电源故障等），而异常则是影响系统正常运行的中断（如除0、越界访问等）。

中断的作用是什么？

中断使得计算机系统具备应对对处理突发事件的能力，提高了CPU的工作效率，如果没有中断系统，CPU就只能按照原来的程序编写的先后顺序，对各个外设进行查询和处理，即轮询工作方式，轮询方法貌似公平，但实际工作效率却很低，却不能及时响应紧急事件。

网络 i/o

你了解过哪些io模型？

- 阻塞I/O模型：应用程序发起I/O操作后会被阻塞，直到操作完成才返回结果。适用于对实时性要求不高的场景。
- 非阻塞I/O模型：应用程序发起I/O操作后立即返回，不会被阻塞，但需要不断轮询或者使用select/poll/epoll等系统调用来检查I/O操作是否完成。适合于需要进行多路复用的场景，例如需要同时处理多个socket连接的服务器程序。
- I/O复用模型：通过select、poll、epoll等系统调用，应用程序可以同时等待多个I/O操作，当其中任何一个I/O操作准备就绪时，应用程序会被通知。适合于需要同时处理多个I/O操作的场景，比如高并发的服务端程序。
- 信号驱动I/O模型：应用程序发起I/O操作后，可以继续做其他事情，当I/O操作完成时，操作系统会向应用程序发送信号来通知其完成。适合于需要异步I/O通知的场景，可以提高系统的并发能力。
- 异步I/O模型：应用程序发起I/O操作后可以立即做其他事情，当I/O操作完成时，应用程序会得到通知。异步I/O模型由操作系统内核完成I/O操作，应用程序只需等待通知即可。适合于需要大量并发连接和高性能的场景，能够减少系统调用次数，提高系统效率。

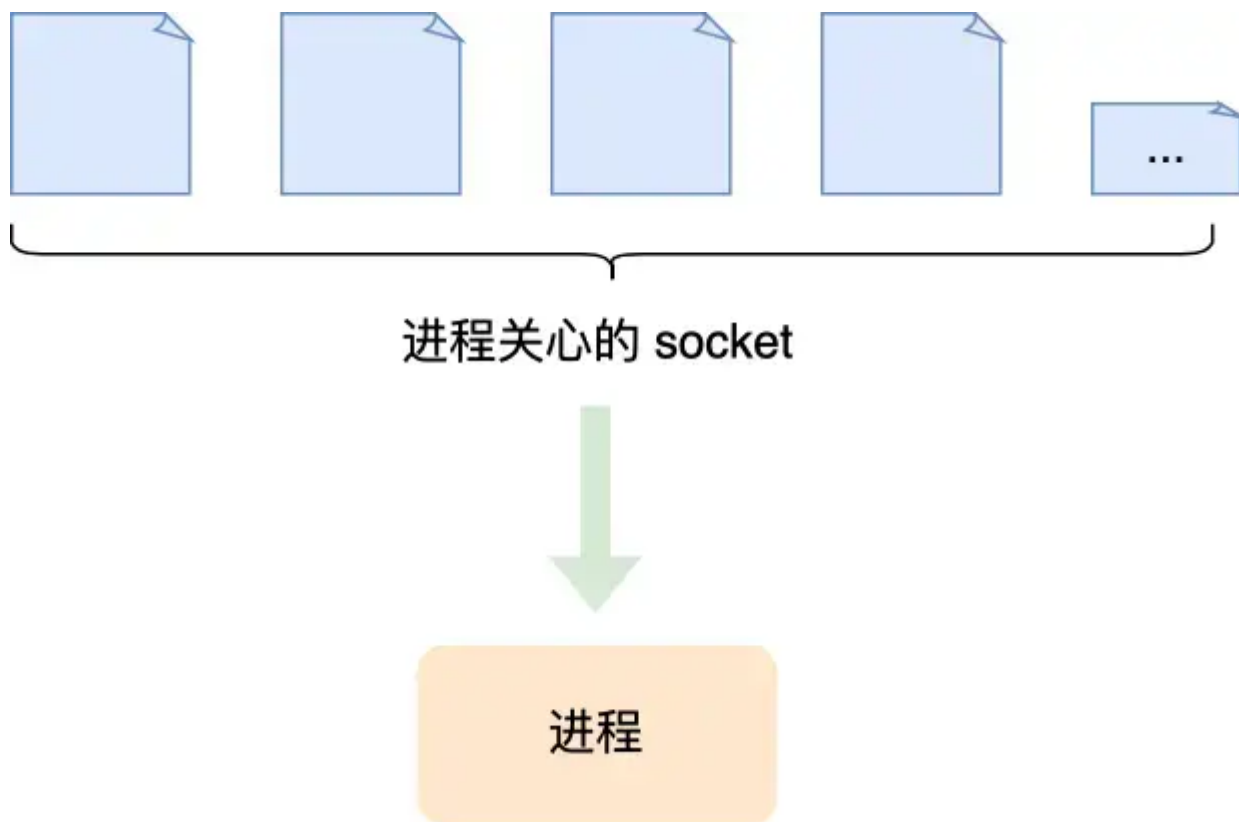
服务器处理并发请求有哪几种方式？

- 单线程web服务器方式：web服务器一次处理一个请求，结束后读取并处理下一个请求，性能比较低，一次只能处理一个请求。
- 多进程/多线程web服务器：web服务器生成多个进程或线程并行处理多个用户请求，进程或线程可以按需或事先生成。有的web服务器应用程序为每个用户请求生成一个单独的进程或线程来进行响应，不过，一旦并发请求数量达到成千上万时，多个同时运行的进程或线程将会消耗大量的系统资源。（即每个进程只能响应一个请求，并且一个进程对应一个线程）
- I/O多路复用web服务器：web服务器可以I/O多路复用，达到只用一个线程就能监听和处理多个客户端的 i/o 事件。

- 多路复用多线程web服务器：将多进程和多路复用的功能结合起来形成的web服务器架构，其避免了让一个进程服务于过多的用户请求，并能充分利用多CPU主机所提供的计算能力。（这种架构可以理解为有多个进程，并且一个进程又生成多个线程，每个线程处理一个请求）

讲一下io多路复用

IO多路复用是一种IO得处理方式，指的是复用一个线程，处理多个socket中的事件。能够资源复用，防止创建过多线程导致的上下文切换的开销。



select、poll、epoll 的区别是什么？

我们熟悉的 select/poll/epoll 内核提供给用户态的多路复用系统调用，**进程可以通过一个系统调用函数从内核中获取多个事件。**

select/poll/epoll 是如何获取网络事件的呢？在获取事件时，先把所有连接（文件描述符）传给内核，再由内核返回产生了事件的连接，然后在用户态中再处理这些连接对应的请求即可。

select/poll/epoll 这是三个多路复用接口，都能实现 C10K 吗？接下来，我们分别说说它们。

select/poll

select 实现多路复用的方式是，将已连接的 Socket 都放到一个**文件描述符集合**，然后调用 select 函数将文件描述符集合**拷贝**到内核里，让内核来检查是否有网络事件产生，检查的方式很粗暴，就是通过**遍历**文件描述符集合的方式，当检查到有事件产生后，将此 Socket 标记为可读或可写，接着再把整个文件描述符集合**拷贝**回用户态里，然后用户态还需要再通过**遍历**的方法找到可读或可写的 Socket，然后再对其处理。

所以，对于 select 这种方式，需要进行 **2 次「遍历」文件描述符集合**，一次是在内核态里，一个次是在用户态里，而且还会发生 **2 次「拷贝」文件描述符集合**，先从用户空间传入内核空间，由内核修改后，再传出到用户空间中。

select 使用固定长度的 Bitmap，表示文件描述符集合，而且所支持的文件描述符的个数是有限制的，在 Linux 系统中，由内核中的 FD_SETSIZE 限制，默认最大值为 1024，只能监听 0~1023 的文件描述符。

poll 不再用 Bitmap 来存储所关注的文件描述符，取而代之用动态数组，以链表形式来组织，突破了 select 的文件描述符个数限制，当然还会受到系统文件描述符限制。

但是 poll 和 select 并没有太大的本质区别，**都是使用「线性结构」存储进程关注的 Socket 集合，因此都需要遍历文件描述符集合来找到可读或可写的 Socket，时间复杂度为 $O(n)$** ，而且也需要在用户态与内核态之间拷贝文件描述符集合，这种方式随着并发数上来，性能的损耗会呈指数级增长。

epoll

先复习下 epoll 的用法。如下的代码中，先用 epoll_create 创建一个 epoll 对象 epfd，再通过 epoll_ctl 将需要监视的 socket 添加到 epfd 中，最后调用 epoll_wait 等待数据。

```
int s = socket(AF_INET, SOCK_STREAM, 0);
bind(s, ...);
listen(s, ...)

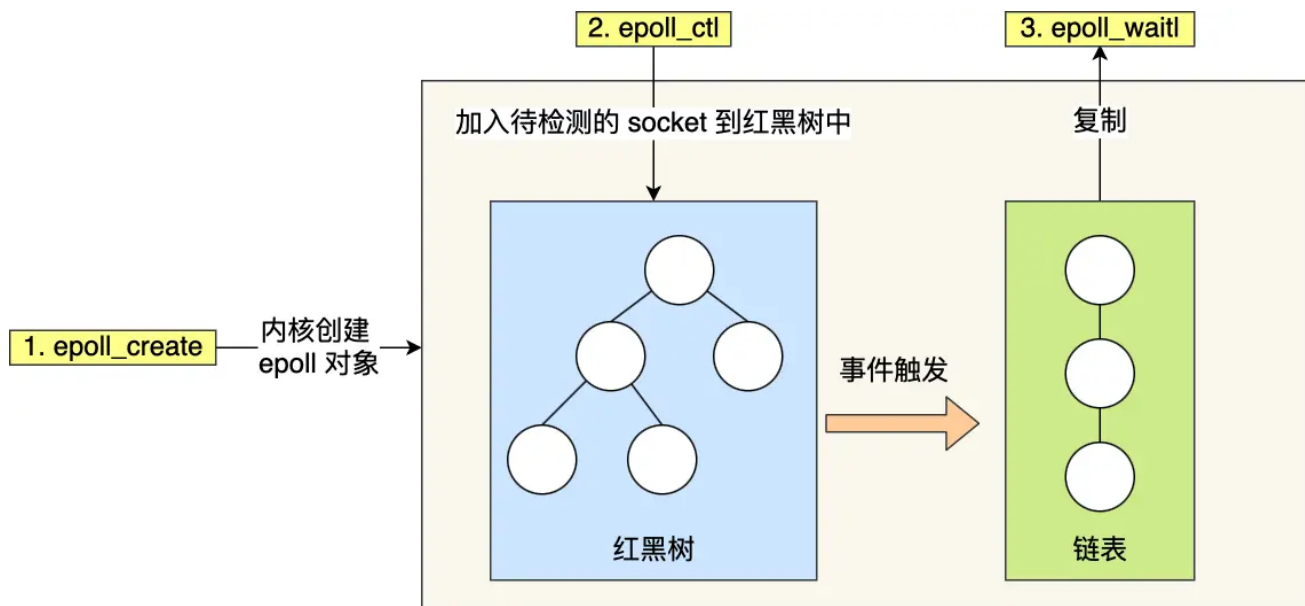
int epfd = epoll_create(...);
epoll_ctl(epfd, ...); //将所有需要监听的socket添加到epfd中

while(1) {
    int n = epoll_wait(...);
    for(接收到数据的socket){
        //处理
    }
}
```

epoll 通过两个方面，很好解决了 select/poll 的问题。

- 第一点，epoll 在内核里使用**红黑树来跟踪进程所有待检测的文件描述符**，把需要监控的 socket 通过 epoll_ctl() 函数加入内核中的红黑树里，红黑树是个高效的数据结构，增删改一般时间复杂度是 $O(\log n)$ 。而 select/poll 内核里没有类似 epoll 红黑树这种保存所有待检测的 socket 的数据结构，所以 select/poll 每次操作时都传入整个 socket 集合给内核，而 epoll 因为在内核维护了红黑树，可以保存所有待检测的 socket，所以只需要传入一个待检测的 socket，减少了内核和用户空间大量的数据拷贝和内存分配。
- 第二点，epoll 使用**事件驱动**的机制，内核里**维护了一个链表来记录就绪事件**，当某个 socket 有事件发生时，通过**回调函数**内核会将其加入到这个就绪事件列表中，当用户调用 epoll_wait() 函数时，只会返回有事件发生的文件描述符的个数，不需要像 select/poll 那样轮询扫描整个 socket 集合，大大提高了检测的效率。

从下图你可以看到 epoll 相关的接口作用：



epoll 的方式即使监听的 Socket 数量越来越多的时候，效率不会大幅度降低，能够同时监听的 Socket 的数目也非常的多了，上限就为系统定义的进程打开的最大文件描述符个数。因而，**epoll 被称为解决 C10K 问题的利器。**

epoll 的边缘触发和水平触发有什么区别？

epoll 支持两种事件触发模式，分别是**边缘触发 (edge-triggered, ET)** 和**水平触发 (level-triggered, LT)**。

这两个术语还挺抽象的，其实它们的区别还是很好理解的。

- 使用边缘触发模式时，当被监控的 Socket 描述符上有可读事件发生时，**服务器端只会从 epoll_wait 中苏醒一次**，即使进程没有调用 read 函数从内核读取数据，也依然只苏醒一次，因此我们程序要保证一次性将内核缓冲区的数据读取完；
- 使用水平触发模式时，当被监控的 Socket 上有可读事件发生时，**服务器端不断地从 epoll_wait 中苏醒，直到内核缓冲区数据被 read 函数读完才结束**，目的是告诉我们有数据需要读取；

举个例子，你的快递被放到了一个快递箱里，如果快递箱只会通过短信通知你一次，即使你一直没有去取，它也不会再发送第二条短信提醒你，这个方式就是边缘触发；如果快递箱发现你的快递没有被取出，它就会不停地发短信通知你，直到你取出了快递，它才消停，这个就是水平触发的方式。

这就是两者的区别，水平触发的意思是只要满足事件的条件，比如内核中有数据需要读，就一直不断地把这个事件传递给用户；而边缘触发的意思是只有第一次满足条件的时候才触发，之后就不会再传递同样的事件了。

如果使用水平触发模式，当内核通知文件描述符可读写时，接下来还可以继续去检测它的状态，看它是否依然可读或可写。所以在收到通知后，没必要一次执行尽可能多的读写操作。

如果使用边缘触发模式，I/O 事件发生时只会通知一次，而且我们不知道到底能读写多少数据，所以在收到通知后应尽可能地读写数据，以免错失读写的机会。因此，我们会**循环**从文件描述符读写数据，那么如果文件描述符是阻塞的，没有数据可读写时，进程会阻塞在读写函数那里，程序就没办法继续往下执行。所以，**边缘触发模式一般和非阻塞 I/O 搭配使用**，程序会一直执行 I/O 操作，直到系统调用（如 read 和 write）返回错误，错误类型为 EAGAIN 或 EWOULDBLOCK。

一般来说，边缘触发的效率比水平触发的效率要高，因为边缘触发可以减少 epoll_wait 的系统调用次数，系统调用也是有一定的开销的，毕竟也存在上下文的切换。

redis, nginx, netty 是依赖什么做的这么高性能？

主要是依赖**Reactor 模式**实现了高性能网络模式，这个是在i/o多路复用接口基础上实现的了网络模型。Reactor 翻译过来的意思是「反应堆」，这里的反应指的是「**对事件反应**」，也就是**来了一个事件，Reactor 就有相对应的反应/响应**。

Reactor 模式主要由 Reactor 和处理资源池这两个核心部分组成，它俩负责的事情如下：

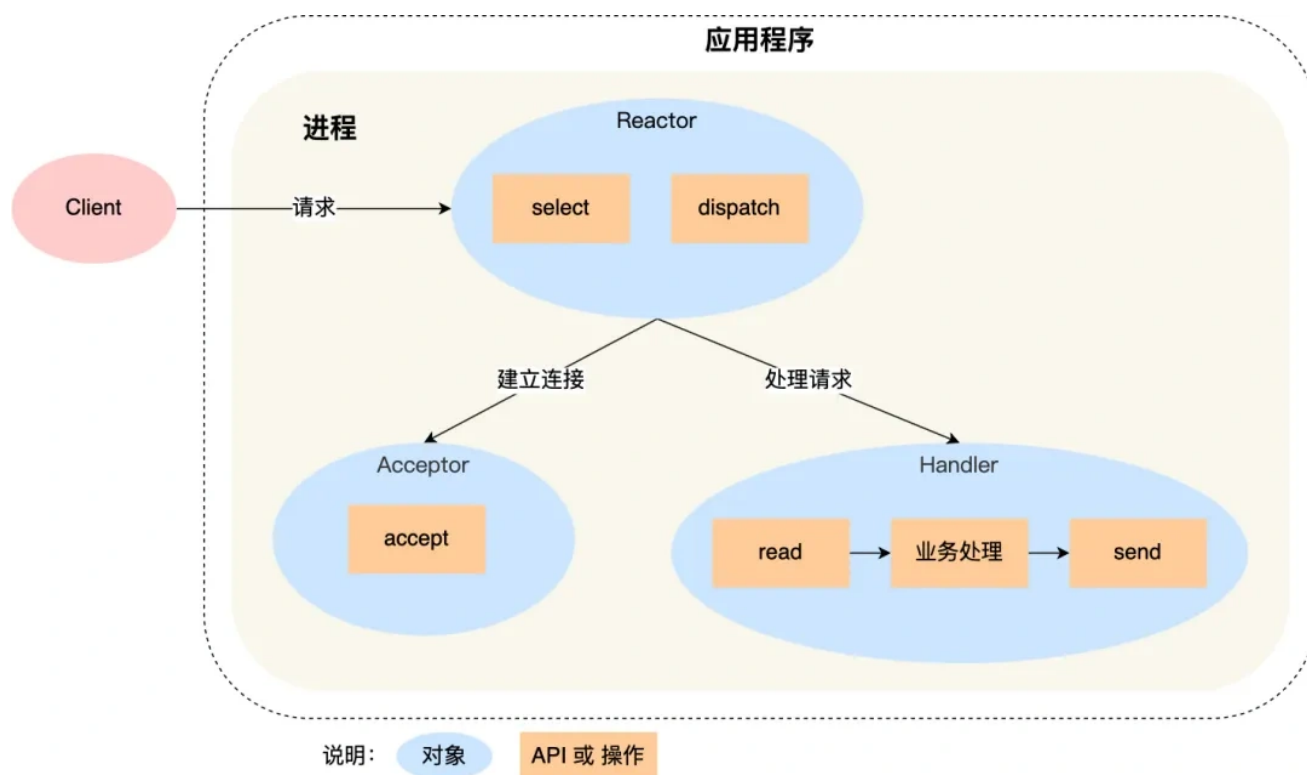
- Reactor 负责监听和分发事件，事件类型包含连接事件、读写事件；
- 处理资源池负责处理事件，如 read -> 业务逻辑 -> send；

Reactor 模式是灵活多变的，可以应对不同的业务场景，灵活在于：

- Reactor 的数量可以只有一个，也可以有多个；
- 处理资源池可以是单个进程 / 线程，也可以是多个进程 / 线程；

Redis

Redis 6.0 之前使用的 Reactor 模型就是单 Reactor 单进程模式。单 Reactor 单进程的方案因为全部工作都在同一个进程内完成，所以实现起来比较简单，不需要考虑进程间通信，也不用担心多进程竞争。



但是，这种方案存在 2 个缺点：

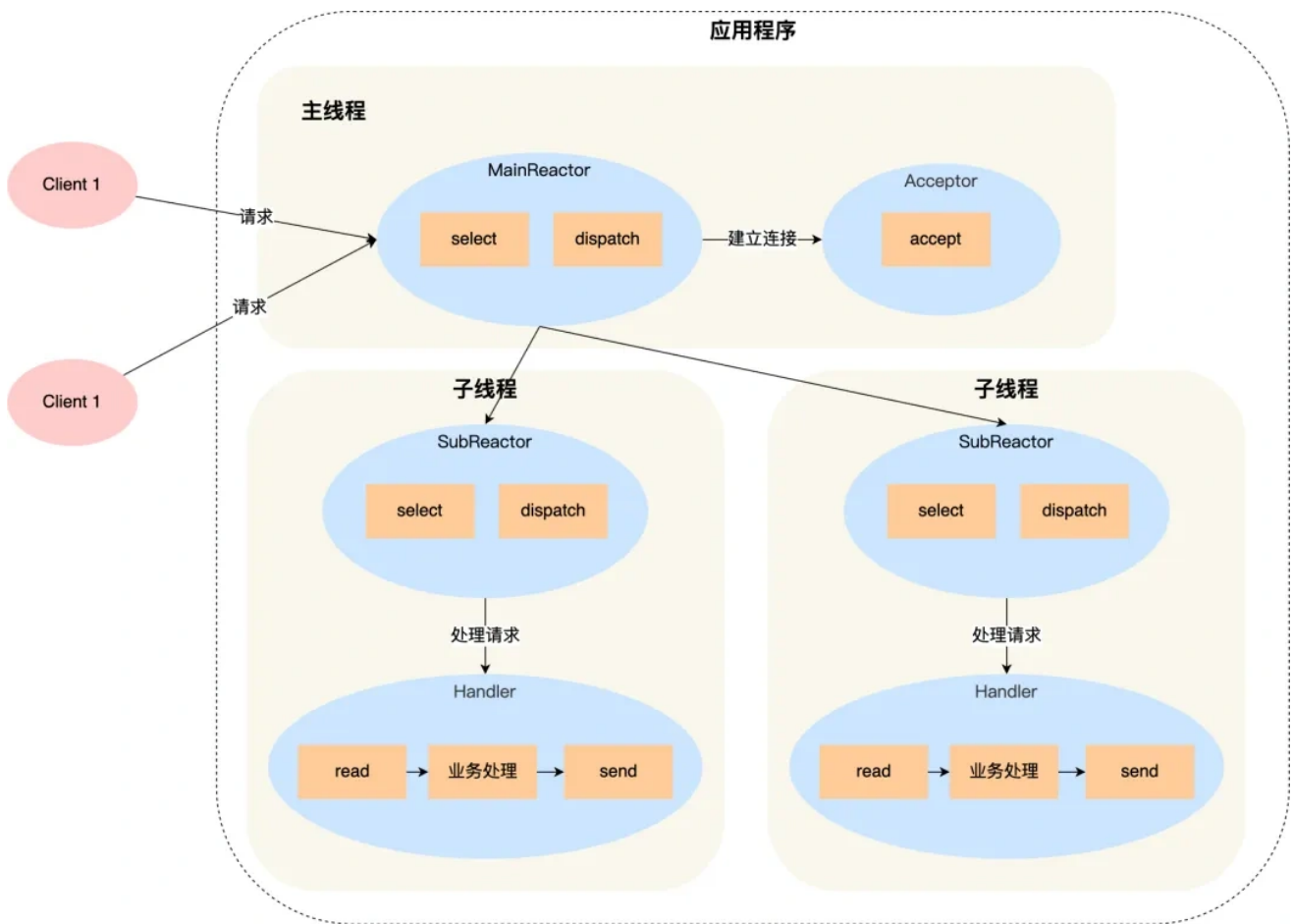
- 第一个缺点，因为只有一个进程，**无法充分利用多核 CPU 的性能**；
- 第二个缺点，Handler 对象在业务处理时，整个进程是无法处理其他连接的事件的，**如果业务处理耗时比较长，那么就造成响应的延迟**；

所以，单 Reactor 单进程的方案**不适用计算机密集型的场景，只适用于业务处理非常快速的场景**。

Redis 是由 C 语言实现的，在 Redis 6.0 版本之前采用的正是「单 Reactor 单进程」的方案，因为 Redis 业务处理主要是在内存中完成，操作的速度是很快的，性能瓶颈不在 CPU 上，所以 Redis 对于命令的处理是单进程的方案。

Netty

Netty 是采用了多 Reactor 多线程方案，如下图：

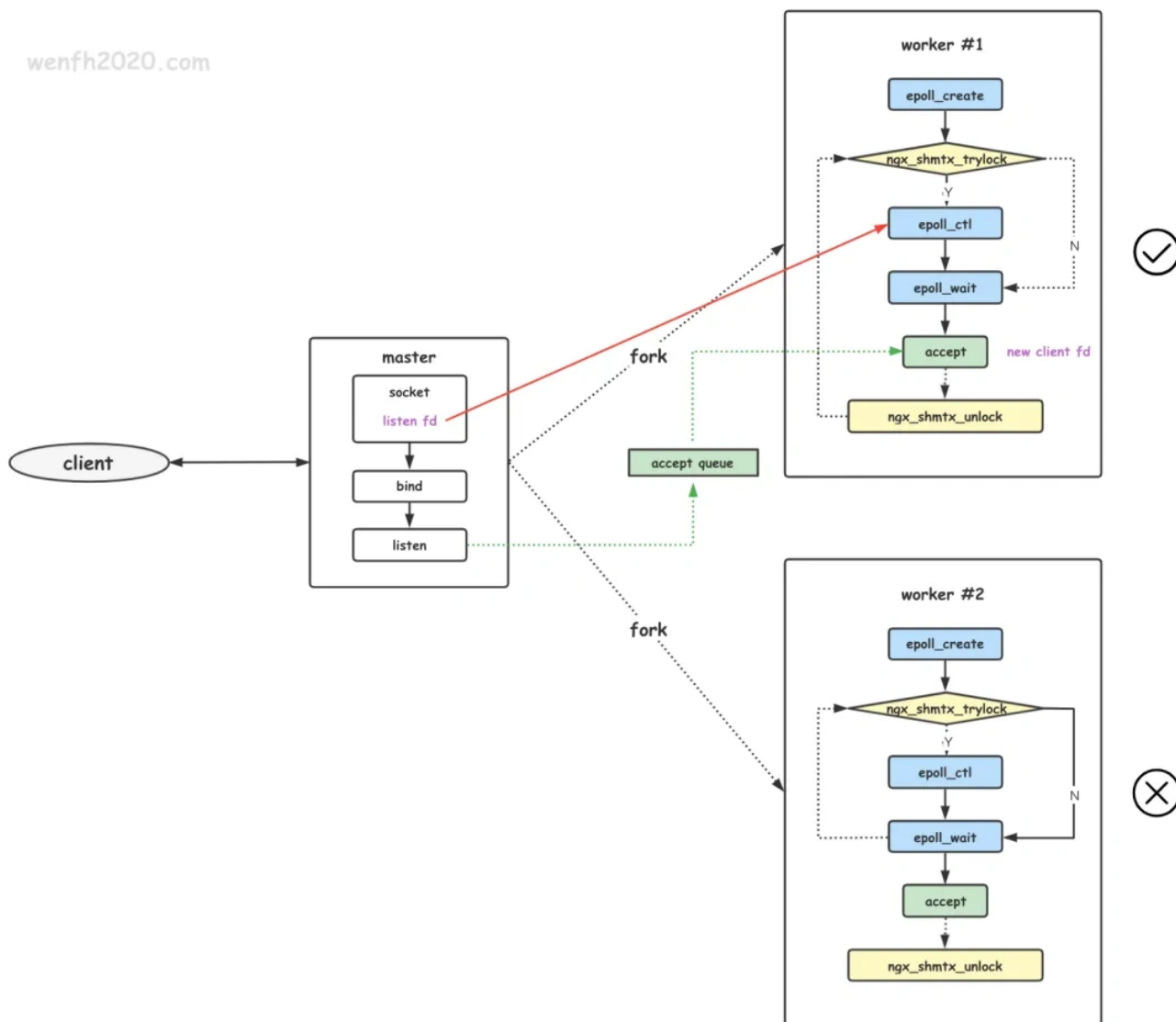


多 Reactor 多线程的方案优势：

- 主线程和子线程分工明确，主线程只负责接收新连接，子线程负责完成后续的业务处理。
- 主线程和子线程的交互很简单，主线程只需要把新连接传给子线程，子线程无须返回数据，直接就可以在子线程将处理结果发送给客户端。

nginx

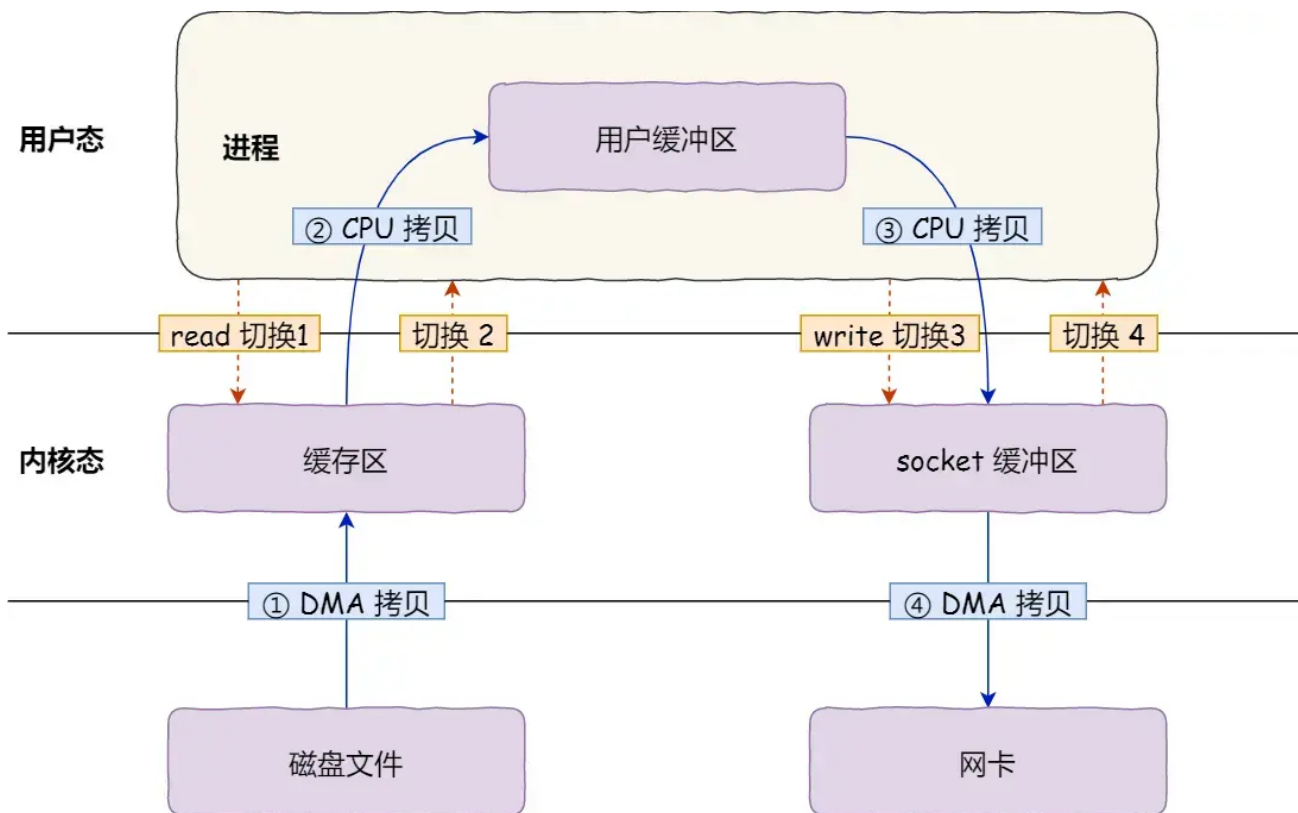
nginx 是多 Reactor 多进程方案，不过方案与标准的多 Reactor 多进程有些差异。



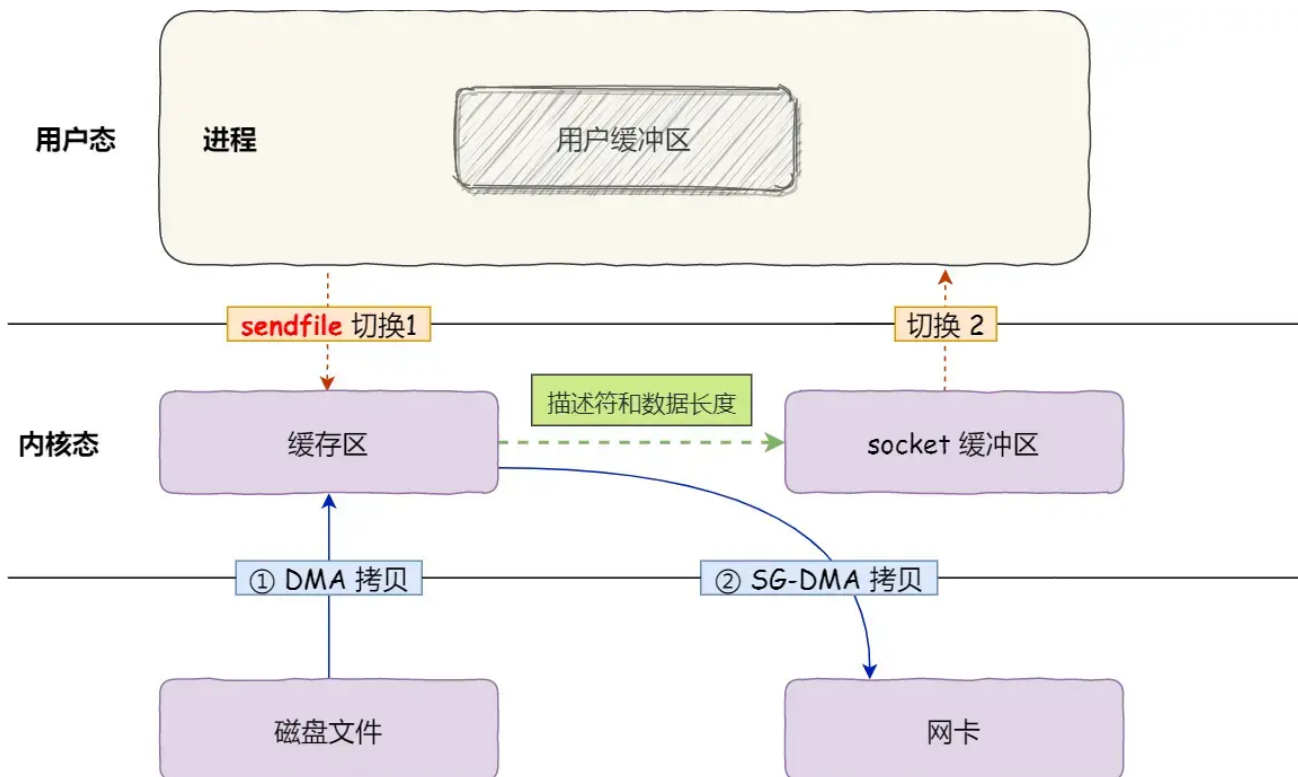
具体差异表现在主进程中仅仅用来初始化 socket，并没有创建 mainReactor 来 accept 连接，而是由子进程的 Reactor 来 accept 连接，通过锁来控制一次只有一个子进程进行 accept（防止出现惊群现象），子进程 accept 新连接后就放到自己的 Reactor 进行处理，不会再分配给其他子进程。

零拷贝是什么？

传统 IO 的工作方式，从硬盘读取数据，然后再通过网卡向外发送，我们需要进行 4 上下文切换，和 4 次数据拷贝，其中 2 次数据拷贝发生在内存里的缓冲区和对应的硬件设备之间，这个是由 DMA 完成，另外 2 次则发生在内核态和用户态之间，这个数据搬移工作是由 CPU 完成的。



为了提高文件传输的性能，于是就出现了零拷贝技术，它通过一次系统调用（sendfile 方法）合并了磁盘读取与网络发送两个操作，降低了上下文切换次数。另外，拷贝数据都是发生在内核中的，天然就降低了数据拷贝的次数。



零拷贝技术的文件传输方式相比传统文件传输的方式，减少了 2 次上下文切换和数据拷贝次数，只需要 2 次上下文切换和数据拷贝次数，就可以完成文件的传输，而且 2 次的数据拷贝过程，都不需要通过 CPU，2 次都是由 DMA 来搬运。

总体来看，零拷贝技术可以把文件传输的性能提高至少一倍以上。



面试鸭 mianshiya.com

程序员面试 刷题神器

八股文一网打尽 面试从未如此轻松

- ✓ 9000+ 中大厂高频面试题
- ✓ 大厂面试官团队原创优质题解
- ✓ 网页版+小程序+IDE、VSCode插件
- ✓ 真题命中率高，持续更新

微信扫一扫 >>

刷高频题拿高薪offer



最新的图解文章都在公众号首发，别忘记关注哦！！如果你想加入百人技术交流群，扫码下方二维码回复「加群」。



扫一扫，关注「小林coding」公众号

图解计算机基础
认准**小林coding**

每一张图都包含小林的认真
只为帮助大家能更好的理解

- ① 关注公众号回复「**图解**」
获取图解系列 PDF
- ② 关注公众号回复「**加群**」
拉你进百人技术交流群